

CSE152B - Spring 2022: Homework 2

Question 1: Warm Up

We will first try SFM using the original implementation from [libviso2](#) (we wrapped it with Python wrappers at [pyviso/src](#)). We will test on a dataset containing 300 images from one sequence of the KITTI dataset with ground-truth camera poses and camera calibration information.

Run the SFM algorithm using the following script. You are required to report two error metrics. The error metric for rotation is defined as the mean of Frobenius norm of the difference between the ground-truth rotation matrix and predicted rotation matrix. The error metric for translation is defined as mean of the L2 distance. Both errors will be printed on the screen as you run the code. **(5 points)**

For Q1: Toggling `if_vis = True/False` allows you to enable/disable the visualization. Disabling the visualization will make the for loop run significantly faster but will not visualize the correspondence/flow and trajectory.

In [1]: *# RUN THIS ONLY ONCE at each start of the notebook*

```
import os
import sys
# change your base path
# os.chdir('./pyviso/') # './'

import viso2
```

In [2]:

```
import os
import numpy as np
np.set_printoptions(precision=4)
np.set_printoptions(suppress=True)
import viso2
import matplotlib.pyplot as plt
from skimage.io import imread
import time

def errorMetric(RPred, RGt, TPred, TGt):
    diffRot = (RPred - RGt)
    diffTrans = (TPred - TGt)
    errorRot = np.sqrt(np.sum(np.multiply(diffRot.reshape(-1), diffRot.reshape(-1))))
    errorTrans = np.sqrt(np.sum(np.multiply(diffTrans.reshape(-1), diffTrans.reshape(-1))))

    return errorRot, errorTrans

# set to True to do the visualization per frame; the images will be saved at '.vis/'
# Turn off if you just want the camera poses and errors
if_vis = True
```

```

# if True the visualization per frame is going to be displayed realtime on screen\
# (only work on a local machine with an active X graphics server); \
# if False there will be no real-time display, but in both options the images will
if_on_screen = False

# parameter settings (for an example, please download

dataset_path = 'dataset_SfM' # On the cluster

img_dir      = os.path.join(dataset_path, 'sequences/00/image_0')
gt_dir       = os.path.join(dataset_path, 'poses/00.txt')
calibFile    = os.path.join(dataset_path, 'sequences/00/calib.txt')
border       = 50;
gap          = 15;

# Load the camera calibration information
with open(calibFile) as fid:
    calibLines = fid.readlines()
    calibLines = [calibLine.strip() for calibLine in calibLines]

calibInfo = [float(calibStr) for calibStr in calibLines[0].split(' ')[1:]]
# param = {'f': calibInfo[0], 'cu': calibInfo[2], 'cv': calibInfo[6]}

# Load the ground-truth depth and rotation
with open(gt_dir) as fid:
    gtTr = [[float(TrStr) for TrStr in line.strip().split(' ')] for line in fid.readlines()]
gtTr = np.asarray(gtTr).reshape(-1, 3, 4)

# param['height'] = 1.6
# param['pitch'] = -0.08
# param['match'] = {'pre_step_size': 64}
first_frame = 0
last_frame = 300
epi = 1e-8

# init visual odometry
params = viso2.MonoParameters()
params.calib.f = calibInfo[0]
params.calib.cu = calibInfo[2]
params.calib.cv = calibInfo[6]
params.height = 1.6
params.pitch = -0.08

first_frame = 0
last_frame = 300

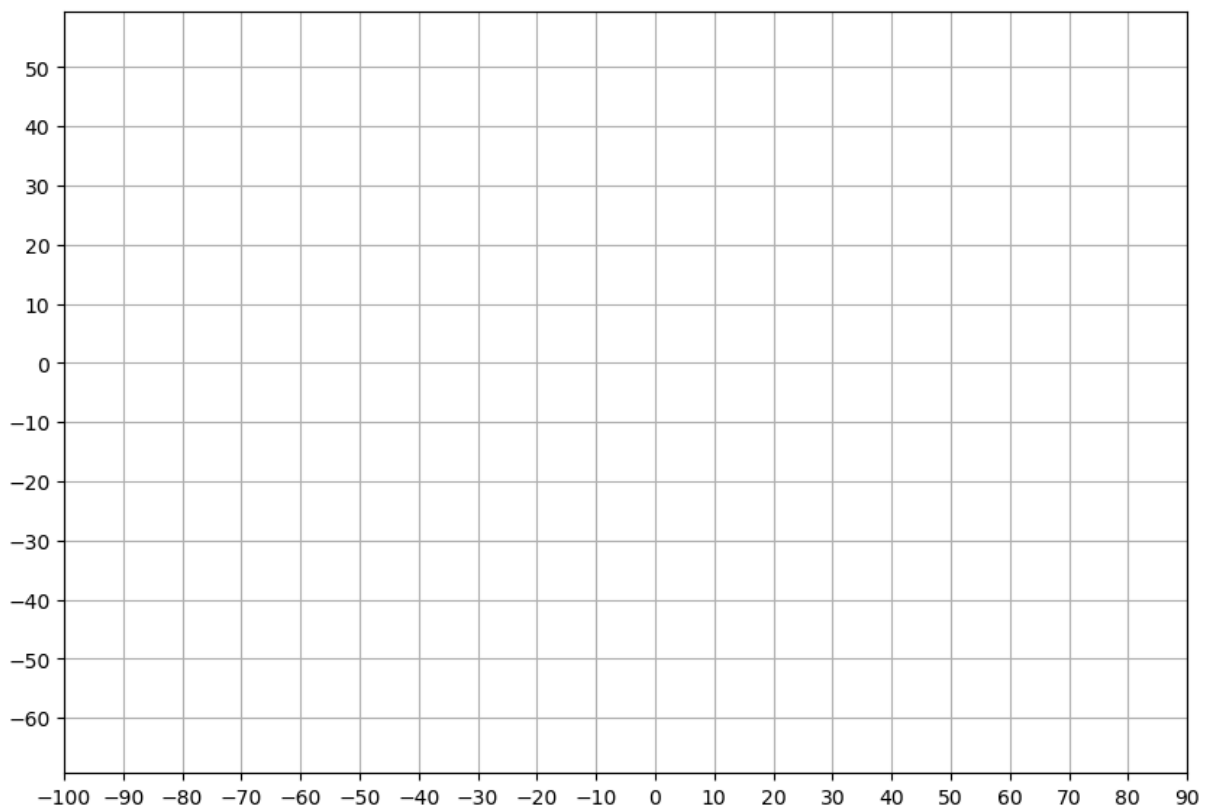
# init transformation matrix array
Tr_total = []
Tr_total_np = []
Tr_total.append(viso2.Matrix.eye(4))
Tr_total_np.append(np.eye(4))

# init viso module
visoMono = viso2.VisualOdometryMono(params)

```

```
if if_vis:
    save_path = 'vis'
    os.makedirs(save_path, exist_ok=True)

    # create figure that is currently EMPTY
    fig = plt.figure(figsize=(10, 15))
    ax1 = plt.subplot(211)
    ax1.axis('off')
    ax2 = plt.subplot(212)
    ax2.set_xticks(np.arange(-100, 100, step=10))
    ax2.set_yticks(np.arange(-500, 500, step=10))
    ax2.axis('equal')
    ax2.grid()
    if if_on_screen:
        plt.ion()
    else:
        plt.ioff()
```



```
In [3]: # for all frames do  
if_replace = False  
errorTransSum = 0  
errorRotSum = 0
```



```

errorRot_list = []
errorTrans_list = []

for frame in range(first_frame, last_frame):
    # 1-based index
    k = frame-first_frame+1

    # read current images
    I = imread(os.path.join(img_dir, '%06d.png'%frame))
    assert(len(I.shape) == 2) # should be grayscale

    # compute egomotion
    process_result = visoMono.process_frame(I, if_replace)
    Tr = visoMono.getMotion()
    matrixer = viso2.Matrix(Tr)
    Tr_np = np.zeros((4, 4))
    Tr.toNumpy(Tr_np) # so awkward...

    # accumulate egomotion, starting with second frame
    if k > 1:
        if process_result is False:
            if_replace = True
            Tr_total.append(Tr_total[-1])
            Tr_total_np.append(Tr_total_np[-1])
        else:
            if_replace = False
            Tr_total.append(Tr_total[-1] * viso2.Matrix.inv(Tr))
            Tr_total_np.append(Tr_total_np[-1] @ np.linalg.inv(Tr_np)) # should be
            print(Tr_total_np[-1])

    # output statistics
    num_matches = visoMono.getNumberOfMatches()
    num_inliers = visoMono.getNumberOfInliers()
    matches = visoMono.getMatches()
    matches_np = np.empty([4, matches.size()])

    for i,m in enumerate(matches):
        matches_np[:, i] = (m.u1p, m.v1p, m.u1c, m.v1c)

    if if_vis:
        # update image
        ax1.clear()
        ax1.imshow(I, cmap='gray', vmin=0, vmax=255)
        if num_matches != 0:
            for n in range(num_matches):
                ax1.plot([matches_np[0, n], matches_np[2, n]], [matches_np[1, n], m
                ax1.set_title('Frame %d'%frame)

        # update trajectory
        if k > 1:
            ax2.plot([Tr_total_np[k-2][0, 3], Tr_total_np[k-1][0, 3]], \
                [Tr_total_np[k-2][2, 3], Tr_total_np[k-1][2, 3]], 'b.-', linewidth=
            ax2.plot([gtTr[k-2][0, 3], gtTr[k-1][0, 3]], \
                [gtTr[k-2][2, 3], gtTr[k-1][2, 3]], 'r.-', linewidth=1)
            ax2.set_title('Blue: estimated trajectory; Red: ground truth trajectory')

```

```

plt.draw()

# Compute rotation
Rpred_p = Tr_total_np[k-2][0:3, 0:3]
Rpred_c = Tr_total_np[k-1][0:3, 0:3]
Rpred = Rpred_c.transpose() @ Rpred_p
Rgt_p = np.squeeze(gtTr[k-2, 0:3, 0:3])
Rgt_c = np.squeeze(gtTr[k-1, 0:3, 0:3])
Rgt = Rgt_c.transpose() @ Rgt_p

# Compute translation
Tpred_p = Tr_total_np[k-2][0:3, 3:4]
Tpred_c = Tr_total_np[k-1][0:3, 3:4]
Tpred = Tpred_c - Tpred_p
Tgt_p = gtTr[k-2, 0:3, 3:4]
Tgt_c = gtTr[k-1, 0:3, 3:4]
Tgt = Tgt_c - Tgt_p

# Compute errors
errorRot, errorTrans = errorMetric(Rpred, Rgt, Tpred, Tgt)
errorRotSum = errorRotSum + errorRot
errorTransSum = errorTransSum + errorTrans
# errorRot_list.append(errorRot)
# errorTrans_list.append(errorTrans)

print('Mean Error Rotation: %.5f'%(errorRotSum / (k-1+epi)))
print('Mean Error Translation: %.5f'%(errorTransSum / (k-1+epi)))

print('== [Result] Frame: %d, Matches %d, Inliers: %.2f'%(frame, num_matches, 1

if if_vis:
    # input('Paused; Press Enter to continue') # Option 1: Manually pause and r
    if if_on_screen:
        plt.pause(0.1) # Or Option 2: enable to this to auto pause for a while
        vis_path = os.path.join(save_path, 'frame%03d.jpg'%frame)
        fig.savefig(vis_path)
        print('Saved at %s'%vis_path)

    if frame % 50 == 0 or frame == last_frame-1:
        plt.figure(figsize=(10, 15))
        plt.imshow(plt.imread(vis_path))
        plt.axis('off')
        plt.show()

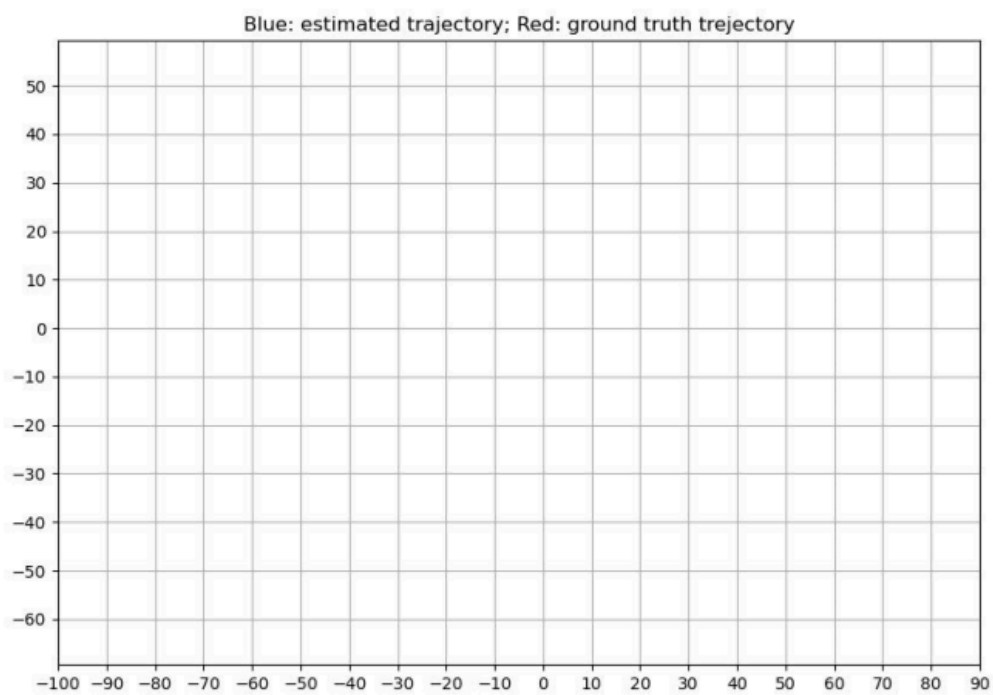
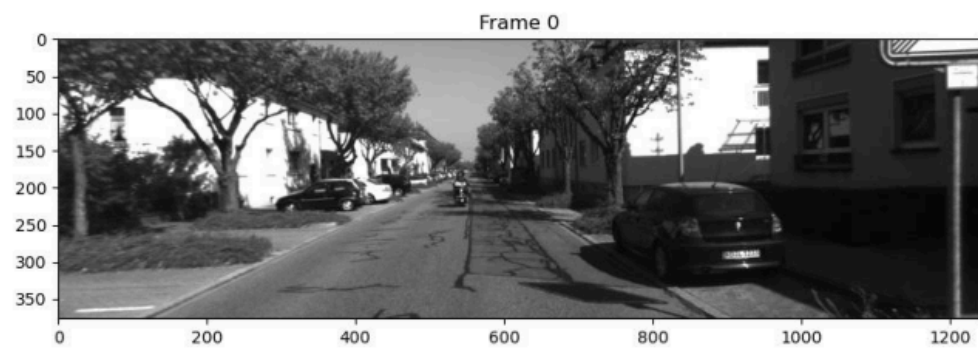
# input('Press Enter to exit')

```

```

Mean Error Rotation: 6947558.55240
Mean Error Translation: 9718750861.05217
== [Result] Frame: 0, Matches 0, Inliers: 0.00
Saved at vis/frame000.jpg
<Figure size 640x480 with 0 Axes>

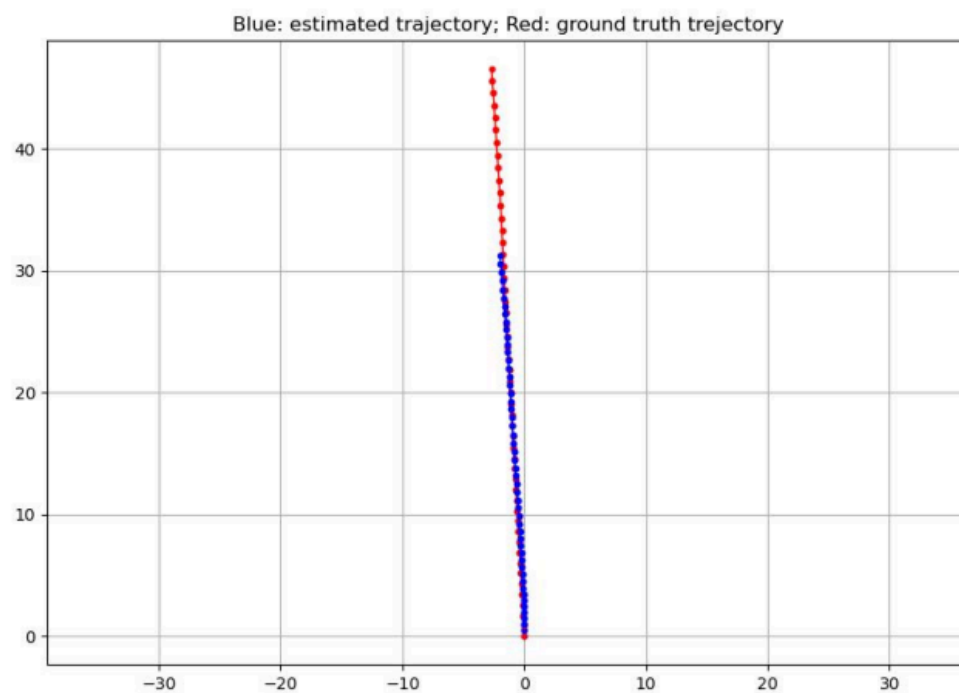
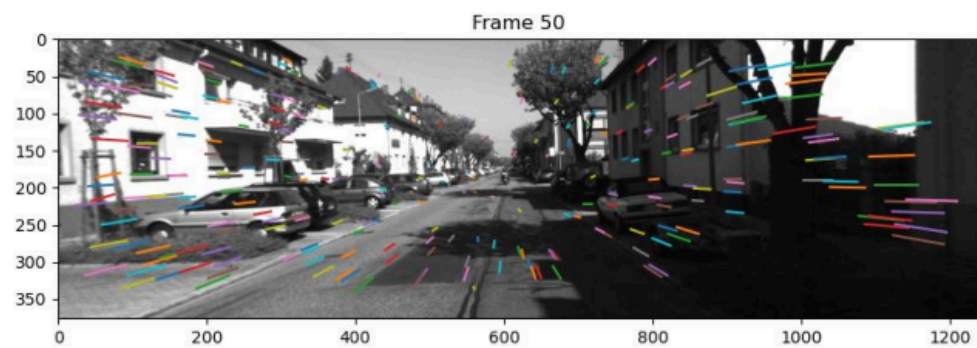
```



```

== [Result] Frame: 45, Matches 263, Inliers: 78.71
Saved at vis/frame045.jpg
Number of match before refinement: 2670
Number of match after refinement: 2248
[[ 0.9968 -0.0128 -0.0791 -1.7282]
 [ 0.0131 0.9999 0.0032 -0.3586]
 [ 0.079 -0.0042 0.9969 28.4473]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00416
Mean Error Translation: 2.42161
== [Result] Frame: 46, Matches 271, Inliers: 81.92
Saved at vis/frame046.jpg
Number of match before refinement: 2752
Number of match after refinement: 2333
[[ 0.9968 -0.0121 -0.0785 -1.794 ]
 [ 0.0122 0.9999 0.0012 -0.3623]
 [ 0.0785 -0.0022 0.9969 29.1674]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00411
Mean Error Translation: 2.37628
== [Result] Frame: 47, Matches 278, Inliers: 80.22
Saved at vis/frame047.jpg
Number of match before refinement: 2830
Number of match after refinement: 2351
[[ 0.9969 -0.0082 -0.0784 -1.8586]
 [ 0.0081 1.      -0.0013 -0.3698]
 [ 0.0784 0.0006 0.9969 29.8567]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00408
Mean Error Translation: 2.33337
== [Result] Frame: 48, Matches 286, Inliers: 81.12
Saved at vis/frame048.jpg
Number of match before refinement: 2851
Number of match after refinement: 2364
[[ 0.9968 -0.0096 -0.0793 -1.919 ]
 [ 0.0096 1.      -0.0012 -0.3803]
 [ 0.0793 0.0004 0.9968 30.5637]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00403
Mean Error Translation: 2.29177
== [Result] Frame: 49, Matches 282, Inliers: 83.33
Saved at vis/frame049.jpg
Number of match before refinement: 2862
Number of match after refinement: 2305
[[ 0.9967 -0.0108 -0.0804 -1.977 ]
 [ 0.0108 0.9999 -0.      -0.3923]
 [ 0.0804 -0.0008 0.9968 31.262 ]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00398
Mean Error Translation: 2.25197
== [Result] Frame: 50, Matches 276, Inliers: 80.07
Saved at vis/frame050.jpg
<Figure size 640x480 with 0 Axes>

```



```

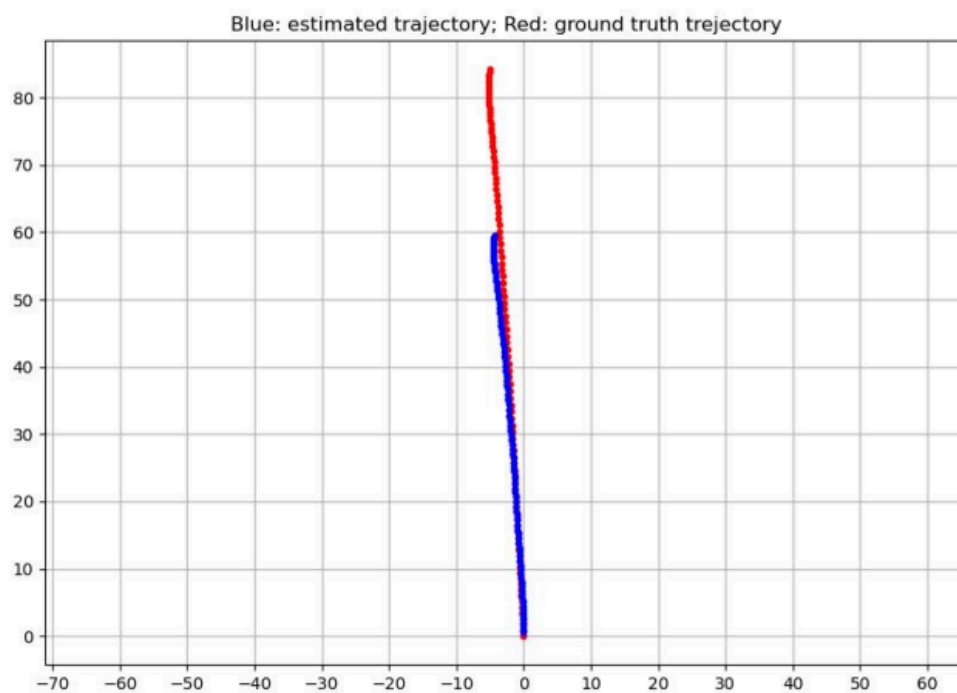
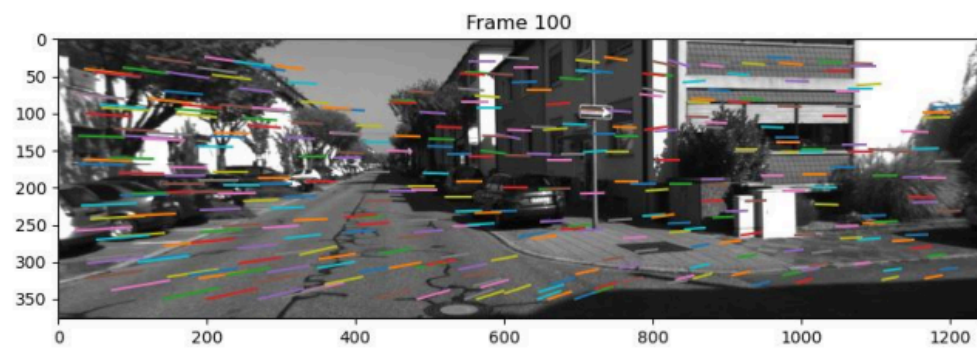
[[ 0.9929 -0.0306 -0.1152 -4.4389]
 [ 0.0293 0.9995 -0.0132 -0.8885]
 [ 0.1155 0.0097 0.9933 56.0629]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00330
Mean Error Translation: 1.34260
== [Result] Frame: 90, Matches 327, Inliers: 85.93
Saved at vis/frame090.jpg
Number of match before refinement: 3485
Number of match after refinement: 3177
[[ 0.9936 -0.0356 -0.1075 -4.4688]
 [ 0.0348 0.9993 -0.0095 -0.9014]
 [ 0.1078 0.0057 0.9942 56.4311]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00328
Mean Error Translation: 1.32964
== [Result] Frame: 91, Matches 325, Inliers: 89.23
Saved at vis/frame091.jpg
Number of match before refinement: 3513
Number of match after refinement: 3200
[[ 0.9947 -0.0335 -0.0972 -4.4995]
 [ 0.0327 0.9994 -0.0095 -0.9091]
 [ 0.0974 0.0062 0.9952 56.7869]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00326
Mean Error Translation: 1.31689
== [Result] Frame: 92, Matches 325, Inliers: 88.00
Saved at vis/frame092.jpg
Number of match before refinement: 3556
Number of match after refinement: 3273
[[ 0.9963 -0.0279 -0.0814 -4.5375]
 [ 0.0272 0.9996 -0.0098 -0.9146]
 [ 0.0816 0.0075 0.9966 57.1355]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00324
Mean Error Translation: 1.30438
== [Result] Frame: 93, Matches 341, Inliers: 90.03
Saved at vis/frame093.jpg
Number of match before refinement: 3573
Number of match after refinement: 3212
[[ 0.9976 -0.0249 -0.0643 -4.5473]
 [ 0.0244 0.9997 -0.0086 -0.9235]
 [ 0.0645 0.007 0.9979 57.4843]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00324
Mean Error Translation: 1.29198
== [Result] Frame: 94, Matches 347, Inliers: 88.18
Saved at vis/frame094.jpg
Number of match before refinement: 3546
Number of match after refinement: 3223
[[ 0.9987 -0.0285 -0.0433 -4.5436]
 [ 0.0282 0.9996 -0.0073 -0.9302]
 [ 0.0435 0.0061 0.999 57.8366]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00324
Mean Error Translation: 1.27967

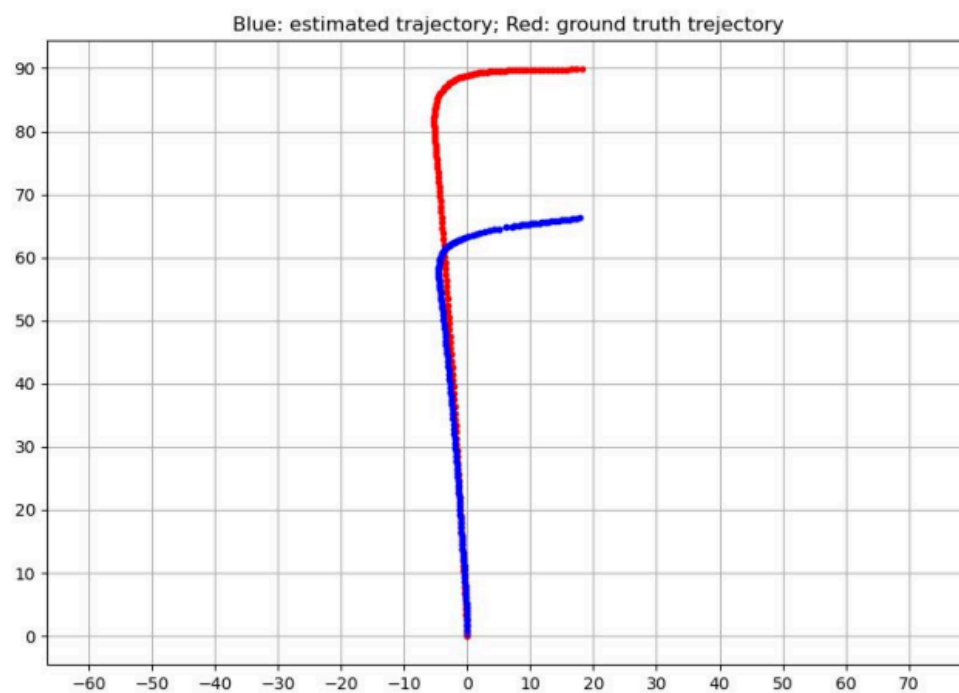
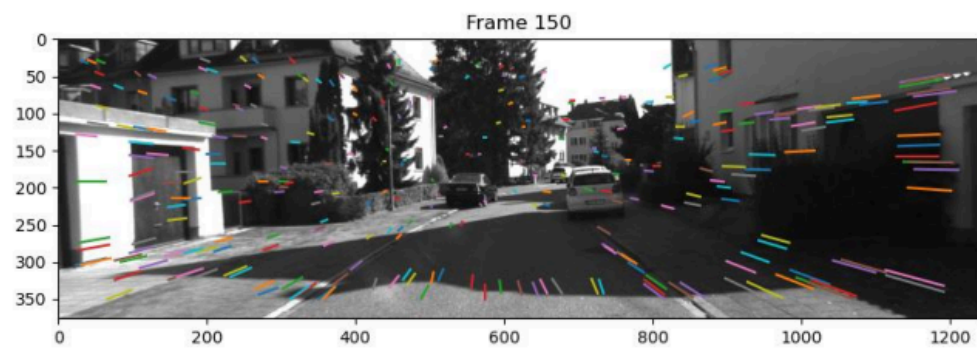
```

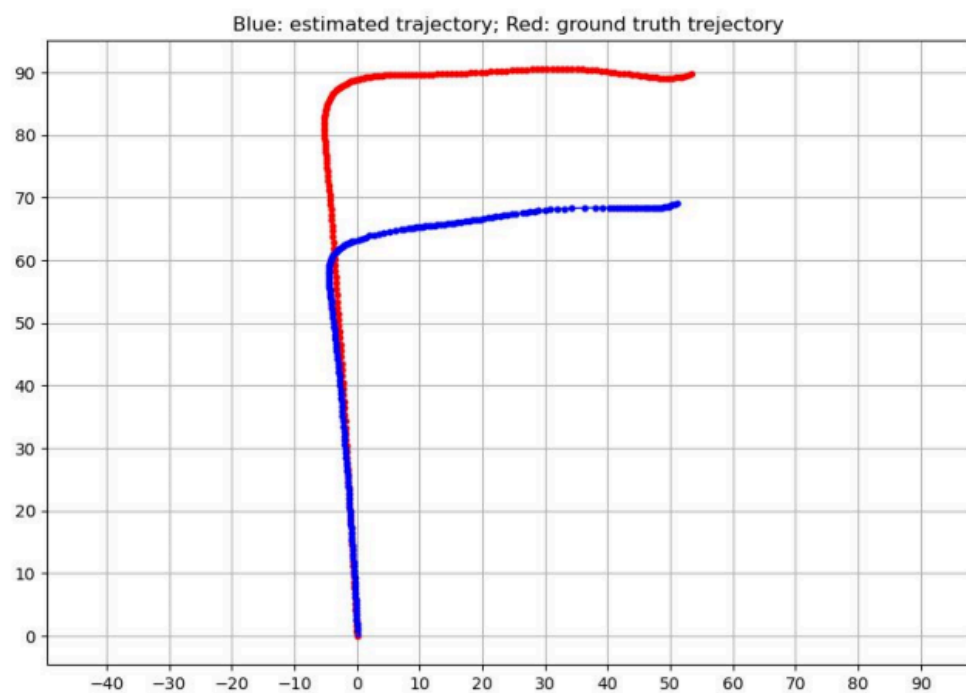
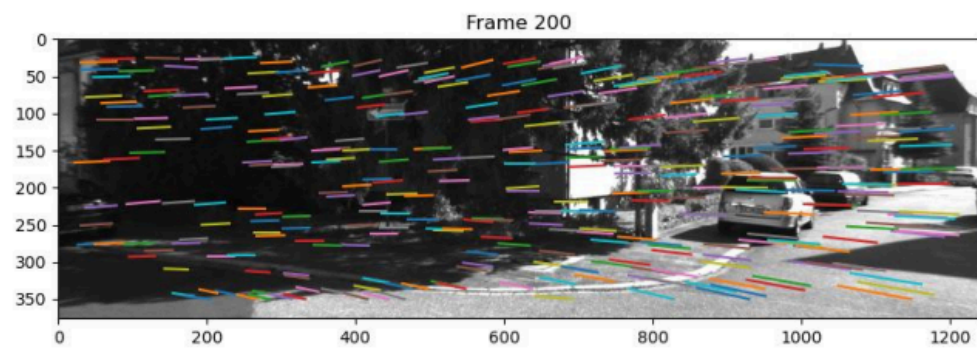
```

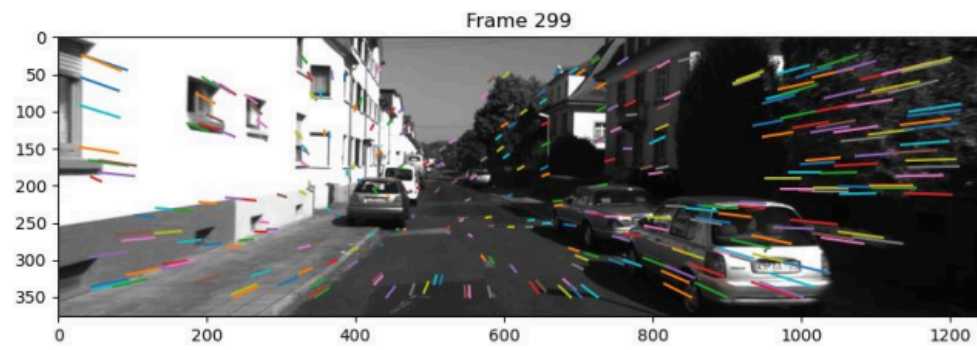
== [Result] Frame: 95, Matches 337, Inliers: 87.24
Saved at vis/frame095.jpg
Number of match before refinement: 3511
Number of match after refinement: 3192
[[ 0.9993 -0.0323 -0.0166 -4.5406]
 [ 0.0322 0.9995 -0.0058 -0.9321]
 [ 0.0168 0.0053 0.9998 58.1764]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00322
Mean Error Translation: 1.26763
== [Result] Frame: 96, Matches 346, Inliers: 89.02
Saved at vis/frame096.jpg
Number of match before refinement: 3527
Number of match after refinement: 3192
[[ 0.9993 -0.0352 0.0119 -4.5154]
 [ 0.0352 0.9994 -0.0032 -0.9385]
 [-0.0117 0.0037 0.9999 58.5102]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00321
Mean Error Translation: 1.25581
== [Result] Frame: 97, Matches 338, Inliers: 87.87
Saved at vis/frame097.jpg
Number of match before refinement: 3575
Number of match after refinement: 3275
[[ 0.9983 -0.0388 0.0426 -4.4674]
 [ 0.0389 0.9992 -0.0012 -0.94   ]
 [-0.0426 0.0029 0.9991 58.8394]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00324
Mean Error Translation: 1.24420
== [Result] Frame: 98, Matches 333, Inliers: 87.09
Saved at vis/frame098.jpg
Number of match before refinement: 3599
Number of match after refinement: 3236
[[ 0.996  -0.0387 0.0807 -4.4269]
 [ 0.0387 0.9992 0.001  -0.9439]
 [-0.0806 0.0021 0.9967 59.1777]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00323
Mean Error Translation: 1.23264
== [Result] Frame: 99, Matches 332, Inliers: 87.95
Saved at vis/frame099.jpg
Number of match before refinement: 3606
Number of match after refinement: 3284
[[ 0.9917 -0.0391 0.1222 -4.3659]
 [ 0.0391 0.9992 0.0029 -0.9481]
 [-0.1223 0.0019 0.9925 59.5015]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00321
Mean Error Translation: 1.22141
== [Result] Frame: 100, Matches 332, Inliers: 84.04
Saved at vis/frame100.jpg
<Figure size 640x480 with 0 Axes>

```









1. Report the final rotation and translation error. **(2 points)**

Final Mean ERROR Rotation: 0.00277
Final Mean ERROR Translation: 0.53721

Then answer the questions below

2. In `libviso2`, the feature points are "bucketed" (`pyviso/src/matcher.cpp : Line285 — 326`), which means in a certain area of region, the number of detected keypoint pairs should be within certain bounds. Why? **(3 points)**

The bucketing or grid grouping is nothing new for feature detection. We do this to reduce redundancies and overlapping points within certain regions, this can also help transform data into categories instead of each point being just a point, but now represents a region or area of the image.

3. We have run SFM on a single camera, which means the scale of translation is unknown. However, as you may have observed, the predicted trajectory is still somehow similar to the ground-truth trajectory. How does `libviso2` handle this ambiguity (`viso_mono.cpp : Line245`)? **(5 points)**

The t matrix is multiplied by the parameter height, then divided by the largest (or best) distance sum to find the new position or trajectory as an estimate of how far to go. It can be thought of as a normalization term based on the largest weighted sum.

4. Briefly explain the RANSAC algorithm used in `libviso2` (`viso_mono.cpp : Line113 — 129`). **(5 points)**

The RANSAC algorithm takes random points from the input and estimates a fundamental matrix, it then sees if these points create an improvement in the model, and if they do, they are labeled as inliers, which are important for the model. The method is used to find the ideal inliers or points that improve the model.

Question 2: Using SIFT [4] for SFM

In the second task, you are required to use keypoints and feature descriptors from SIFT for SFM. The SIFT implementation can be found in directory `SIFT`.

(A) Go to `./pyviso/SIFT` directory and run `runSIFT.py` (e.g. `python runSIFT.py --input ../../dataset_SfM/sequences/00/image_0/`). You will save the detected keypoints and feature descriptors under the directory `SIFT`. For image `000abc.png`, the pre-computed features and keypoints should be saved in a `.npy` file named as

`pyviso/SIFT/000abc_feature.npy`. The variable should be a $130 \times N$ matrix with single precision, where N is the number of feature points being detected. For each 130-dimensional feature vector, the first two dimensions are the location of the keypoints (column number first and then row number) on the image plane and the last 128 dimensions are the feature descriptor.

If you enabled visualization, the visualized correspondences and trajectory will be saved to `vis_preFeature/frame{}.jpg`.

(B) Run the following script

```
In [5]: import pyviso.runFeature as runFeature
dataset_path = './dataset_SfM'
feature_dir = 'pyviso/SIFT'

runFeature.runSfM(dataset_path, feature_dir )
```

Start push back!

replace: 0

num: 3206

Feature size: 130

Finish pushing back!

Finish matching features!

Finish bucketing features!

Finish getting matches!

Mean Error Rotation: inf

Mean Error Translation: inf

== [Result] Frame: 0, Matches 0, Inliers: 0.00

/home/dabrin/CSE152B-HW/HW2/pyviso/runFeature.py:176: RuntimeWarning: divide by zero encountered in scalar divide

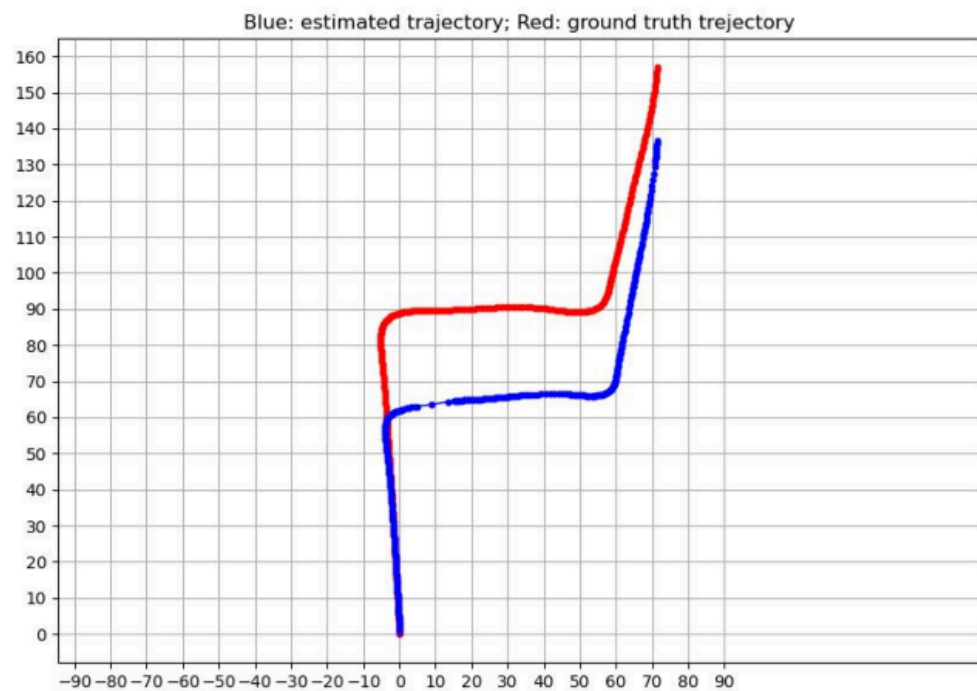
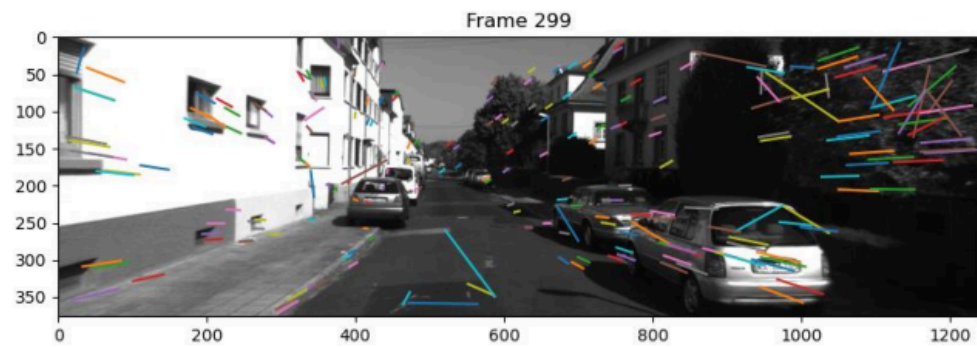
print('Mean Error Rotation: %.5f'%(errorRotSum / (k-1)))

/home/dabrin/CSE152B-HW/HW2/pyviso/runFeature.py:177: RuntimeWarning: divide by zero encountered in scalar divide

print('Mean Error Translation: %.5f'%(errorTransSum / (k-1)))

Saved at vis_preFeature/frame000.jpg

```
Feature size: 130
Finish pushing back!
Start computing pairs!
Number of match before refinement: 918
Number of match after refinement: 918
Finish matching features!
Finish bucketing features!
Finish getting matches!
[[ 0.9948  0.062  0.0813  71.261 ]
 [ -0.0592  0.9976 -0.0364 -4.5903]
 [ -0.0834  0.0314  0.996  135.9742]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00228
Mean Error Translation: 0.57087
== [Result] Frame: 298, Matches 233, Inliers: 82.40
Saved at vis_preFeature/frame298.jpg
Start push back!
replace: 0
num: 1446
Feature size: 130
Finish pushing back!
Start computing pairs!
Number of match before refinement: 901
Number of match after refinement: 901
Finish matching features!
Finish bucketing features!
Finish getting matches!
[[ 0.9951  0.0654  0.0736  71.2962]
 [ -0.0633  0.9975 -0.031  -4.6159]
 [ -0.0754  0.0262  0.9968  136.6078]
 [ 0.      0.      0.      1.    ]]
Mean Error Rotation: 0.00227
Mean Error Translation: 0.56962
== [Result] Frame: 299, Matches 222, Inliers: 81.08
Saved at vis_preFeature/frame299.jpg
<Figure size 640x480 with 0 Axes>
```



1. Report the final rotation and translation error. **(2 points)**

Mean Error Rotation: 0.00227

Mean Error Translation: 0.56962

Next, answer the following questions:

2. Does SIFT yield higher accuracy than the original `libviso2`? Why or why not? If not, can you suggest one possible way to improve? **(5 points)**

SIFT yields slightly higher Rotational accuracy while having lower translational accuracy. It achieves the higher rotational accuracy by making sure the algorithm has rotational invariance by aligning the features it found (described by multidimensional gradient vectors) to the dominant direction, meaning features are found then rotated in matching during the SIFT algorithm. It doesn't have the same method for normalizing the scale, it just uses the classic sift gaussian smoothing at different scales to create the scale invariance but doesn't help with measuring distances which the terms in the previous question described. To improve the translational accuracy, we could add the normalization terms to the new position that were explained from the `viso_mono.cpp` file.

3. Explain how SIFT achieves invariance to a. illumination b. rotation c. scale **(3 points)**

- a. SIFT achieves invariance to illumination by using the scale gradients of the pixels to describe the features, these normalized gradients are much less impacted by illumination changes in small areas as they are multidimensional vectors and normalized based on their region.
- b. SIFT achieves rotational invariance through their histogram method of finding the dominant direction for their gradients, this keeps their frames aligned even in the case of rotation in the image as their feature points are rotated based on the found direction.
- c. Finally it is Scale-Invariant (as suggested in the name Scale Invariant Feature Transform) because of the gaussians used to find the gradients, each feature found should have the same or similar vector descriptors at different scales

Question 3: Using SPyNet [5] for SFM

Now we will compute camera motion from optical flow computed using SPyNet. We first uniformly sample points in an image, then consider the flow-displaced point in the other image as a match. A modified PyTorch implementation of SPyNet is provided in directory `Flow`.

(A) Go to `pyviso/Flow` and run `demo_spynet.py` (e.g. `python demo_spynet.py --input ../../dataset_SfM/sequences/00/image_0`). Try starting with processing only one sample via `--last_frame 1`. If it works fine, remove it and run inference over all images.

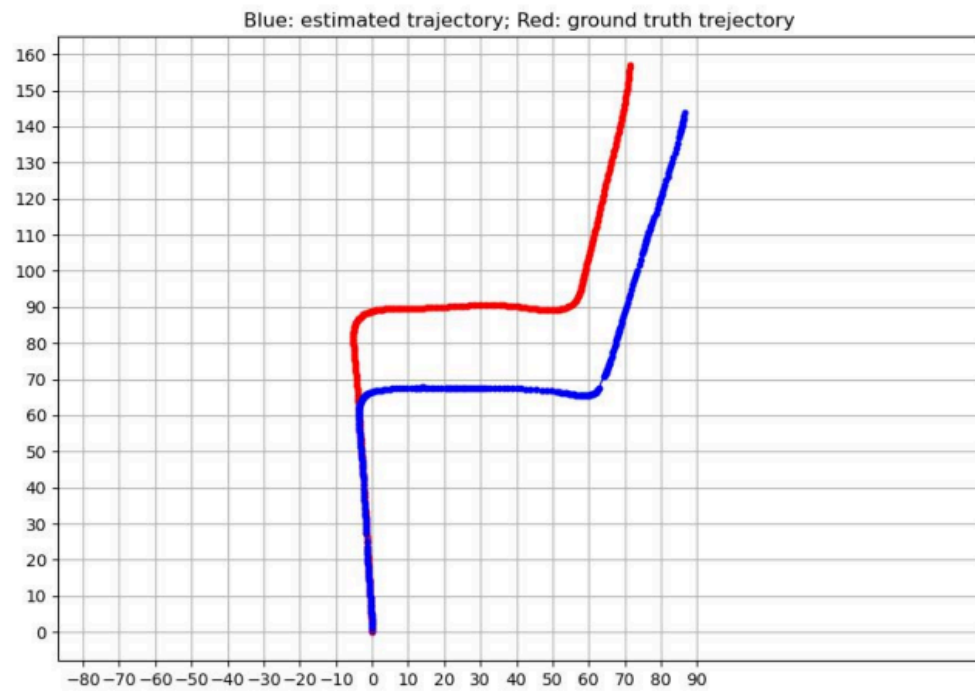
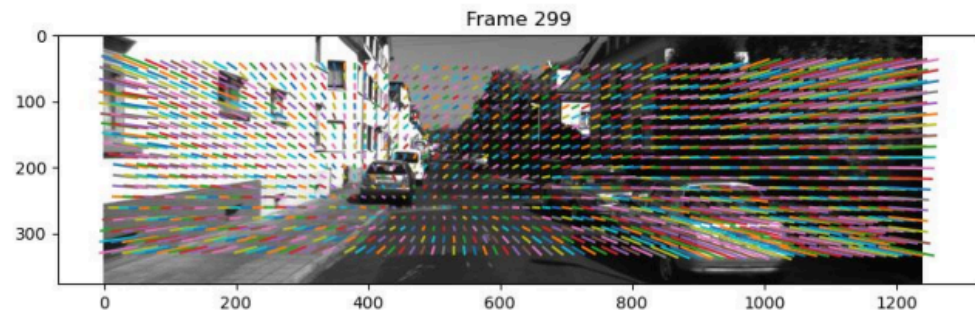
(B) Run the following script.

```
In [6]: import pyviso.runMatch as runMatch
dataset_path = './dataset_SfM'
feature_dir = 'pyviso/Flow'
runMatch.runSFM(dataset_path, feature_dir )
```

```

Saved at vis_preFeature/frame295.jpg
(1463, 4)
[[ 0.963    0.1763   0.2038  86.1552]
 [ -0.1575   0.9819  -0.1055 -19.7766]
 [ -0.2187   0.0695   0.9733 141.8602]
 [ 0.         0.         0.         1.        ]]
Mean Error Rotation: 0.00450
Mean Error Translation: 0.61120
== [Result] Frame: 296, Matches 1463, Inliers: 95.22
<Figure size 640x480 with 0 Axes>
Saved at vis_preFeature/frame296.jpg
(1463, 4)
[[ 0.9653    0.1723   0.1962  86.2703]
 [ -0.1546   0.9827  -0.1025 -19.8362]
 [ -0.2105   0.0686   0.9752 142.4271]
 [ 0.         0.         0.         1.        ]]
Mean Error Rotation: 0.00451
Mean Error Translation: 0.61000
== [Result] Frame: 297, Matches 1463, Inliers: 96.65
<Figure size 640x480 with 0 Axes>
Saved at vis_preFeature/frame297.jpg
(1463, 4)
[[ 0.9667    0.1715   0.1899  86.3842]
 [ -0.1557   0.9832  -0.0953 -19.9025]
 [ -0.203    0.0625   0.9772 143.0553]
 [ 0.         0.         0.         1.        ]]
Mean Error Rotation: 0.00451
Mean Error Translation: 0.60865
== [Result] Frame: 298, Matches 1463, Inliers: 88.11
<Figure size 640x480 with 0 Axes>
Saved at vis_preFeature/frame298.jpg
(1463, 4)
[[ 0.9682    0.1708   0.1828  86.5076]
 [ -0.1566   0.9836  -0.0895 -19.9712]
 [ -0.1951   0.0581   0.9791 143.7586]
 [ 0.         0.         0.         1.        ]]
Mean Error Rotation: 0.00451
Mean Error Translation: 0.60710
== [Result] Frame: 299, Matches 1463, Inliers: 90.02
<Figure size 640x480 with 0 Axes>
Saved at vis_preFeature/frame299.jpg

```



1. Report the final rotation and translation error. **(2 points)**

Mean Error Rotation: 0.00451

Mean Error Translation: 0.60710

Next, answer the following questions:

2. Does SPyNet yield higher accuracy than the original `libviso2`? Why or why not? If not, can you suggest one possible step to improve? **(5 points)**

SPyNet yields LOWER accuracy than the original and the SIFT matching pipeline as well. The SPyNet model uses the different levels of convolution (similar to sift) but uses a coarse to fine method which can lose some data. It also only calculates the residual flows as they calculate flow as each point for each frame because of the CNN structures in the model. Since the model doesn't match features, but instead each point, it might be harder to see movement of features or pixels through a transformation, we could improve this by adding some spatial transformation matching and similar normalization terms for the translational part that we suggested for SIFT to get a better reading for the size of the transformation.

3. Explain how SPyNet achieves accurate flow with significantly lower computational cost. **(3 points)**

SPyNet achieves this flow with lower cost through its use of CNNs to find motion at lower resolution or downsampled images at the smaller part of the pyramid in the scale space, it uses the coarse to fine method which is less computationally heavy, and because it recalculates flow at each frame and each level, it only calculates residual flow instead of the total flow to reduce the computations needed at each step.

Question 4: Optical Flow with Lucas-Kanade (LK) [7]

In this question you are going to implement the Lucas-Kanade algorithm and test it on a image sequence.

1. Implement the basic Lucas-Kanade method for estimating optical flow. The functions `def extract_window` and `def LucasKanadeOpticalFlow` need to be completed and they can then be used for the main script in the second block (only the code corresponding to `LKmode == 'none'` is needed for questions 4.1 to 4.3). **(5 points)**

answer in the code block below

```
In [3]: import numpy as np
import matplotlib.pyplot as plt

def grayscale(img):
```

```

'''
Converts RGB image to Grayscale
'''

gray=np.zeros((img.shape[0],img.shape[1]))
gray=img[:, :, 0]*0.2989+img[:, :, 1]*0.5870+img[:, :, 2]*0.1140
return gray

def plot_optical_flow(img,U,V,titleStr, if_gray_scale=False):
'''
Plots optical flow given U,V and one of the images
'''

# Change t if required, affects the number of arrows
# t should be between 1 and min(U.shape[0],U.shape[1])
t = 10

# Subsample U and V to get visually pleasing output
U1 = U[:, :t, :t]
V1 = V[:, :t, :t]

# Create meshgrid of subsampled coordinates
r, c = img.shape[0],img.shape[1]
cols,rows = np.meshgrid(np.linspace(0,c-1,c), np.linspace(0,r-1,r))
cols = cols[:, :t, :t]
rows = rows[:, :t, :t]

# Plot optical flow
plt.figure(figsize=(10,10))
if if_gray_scale:
    plt.imshow((img*255.).astype(np.uint8), cmap='gray', vmin=0, vmax=255)
else:
    plt.imshow(img)
plt.quiver(cols,rows,U1,V1)
plt.title(titleStr)
plt.show()

def extract_window(img, x, y, window_size):
'''
The functions takes a grayscale image, a center pixel (x, y), and a half window
outputs a patch of the images of size [window_size*2+1, window_size*2+1]
'''

##### (Q4.1) YOUR CODE HERE #####
x1 = x - window_size
x2 = x + window_size + 1
y1 = y - window_size
y2 = y + window_size + 1
#####

window = img[x1 : x2, y1 : y2]
assert window.shape == (window_size*2+1, window_size*2+1)
coords = (x1, x2, y1, y2)

return window

def LucasKanadeOpticalFlow(I1g, I2g, window_size, u_prev=None, v_prev=None, LKmode=

```

```

'''
Implement the Lucas-Kanade algorithm
Inputs: the two images, window size
Returns: u, v - the optical flow
'''

import numpy as np
from scipy import signal

kernel_x = np.array([[ -1., 1.], [ -1., 1.]])
kernel_y = np.array([[ 1., 1.], [ -1., -1.]])
kernel_t = np.array([[ 1., 1.], [ 1., 1.]]) *.25
w = int(np.ceil(window_size/2)) # window_size is odd, all the pixels with offset
I1g = I1g / 255. # normalize pixels
I2g = I2g / 255. # normalize pixels

# Implement Lucas Kanade for each point, calculate I_x, I_y, I_t
mode = 'same'
fx = signal.convolve2d(I1g, kernel_x, boundary='symm', mode=mode)
fy = signal.convolve2d(I1g, kernel_y, boundary='symm', mode=mode)
ft = signal.convolve2d(I2g, kernel_t, boundary='symm', mode=mode) + \
     signal.convolve2d(I1g, -kernel_t, boundary='symm', mode=mode) # I_t with Gaussian Smoothing
#ft = I2g - I1g # Vanilla I_t without Gaussian Smoothing

u = np.zeros(I1g.shape)
v = np.zeros(I1g.shape)

# within window window_size * window_size
for i in range(w, I1g.shape[0]-w):
    for j in range(w, I1g.shape[1]-w):
        Ix = extract_window(fx, i, j, w).flatten()

        if LKmode == 'none':
            Iy = extract_window(fy, i, j, w).flatten()

        elif LKmode == 'multiscale':
            assert u_prev is not None and v_prev is not None
            assert v_prev.shape == I1g.shape
            ...

            Use i, and u_prev or v_prev to get the accumulated flow i_accu in t
            Use j, and u_prev or v_prev to get the accumulated flow j_accu in t
            Make sure i_accu, j_accu does not go out of bound (i.e. i in range(
            ...

            #_accu = # (Q4.4) YOUR CODE HERE
            #j_accu = # (Q4.4) YOUR CODE HERE
            i_accu = np.clip(i + v_prev[i, j], w, I1g.shape[0]-w-1).astype(np.int)
            j_accu = np.clip(j + u_prev[i, j], w, I1g.shape[1]-w-1).astype(np.int)
            Iy = extract_window(fy, i_accu, j_accu, w).flatten()

        else:
            raise (RuntimeError('Invalid LK mode!'))

        It = extract_window(ft, i, j, w).flatten()

#... # (Q4.4) YOUR CODE HERE, based on the math on slide #45 of Lecture
# ... # (Q4.4) YOUR CODE HERE, based on the math on slide #45 of Lecture

```

```

b = -It
A = np.vstack((Ix, Iy)).T

ws, _ = np.linalg.eig(A.transpose() @ A)
lam = 1e-5
tau = 1e-6

# if threshold  $\tau$  is larger than the smallest eigenvalue of  $A^T A$ :
if ws[1] > tau:
    # (Q4.1) YOUR CODE HERE: get velocity nu based on  $(A^T A + \text{lam} * \text{np.eye}(2))$ 
    nu = np.linalg.solve(A.T @ A + lam * np.eye(2), A.T @ b)
    u[i,j]=nu[0]
    v[i,j]=nu[1]
else:
    u[i,j]=0.
    v[i,j]=0.

return u, v

```

2. Plot standard Lucas-Kanade optical flow (that is, with LKmode as 'none') for the three choices for the pair of images im1 and im2 specified below, using window sizes of 13. For each plot, comment on whether and how the estimated flow matches your intuition given the motion visible for the image pair. **(6 points)**

```

In [9]: images = []
for i in range(1,5):
    images.append(plt.imread('OpticalFlowImages/im'+str(i)+'.png')[:, :288, :])
# each image after converting to gray scale is of size -> 400x288
window = 13
U, V = LucasKanadeOpticalFlow(grayScale(images[0]), grayScale(images[0]), window)
plot_optical_flow(images[0], U, V, 'image2 = ' + str(0) + ', window = ' + str(window))

window = 13
U, V = LucasKanadeOpticalFlow(grayScale(images[0]), grayScale(images[1]), window)
plot_optical_flow(images[1], U, V, 'image2 = ' + str(1) + ', window = ' + str(window))

window = 13
U, V = LucasKanadeOpticalFlow(grayScale(images[0]), grayScale(images[2]), window)
plot_optical_flow(images[2], U, V, 'image2 = ' + str(2) + ', window = ' + str(window))

window = 13
U, V = LucasKanadeOpticalFlow(grayScale(images[0]), grayScale(images[3]), window)
plot_optical_flow(images[3], U, V, 'image2 = ' + str(3) + ', window = ' + str(window))

```

image2 = 0, window = 13

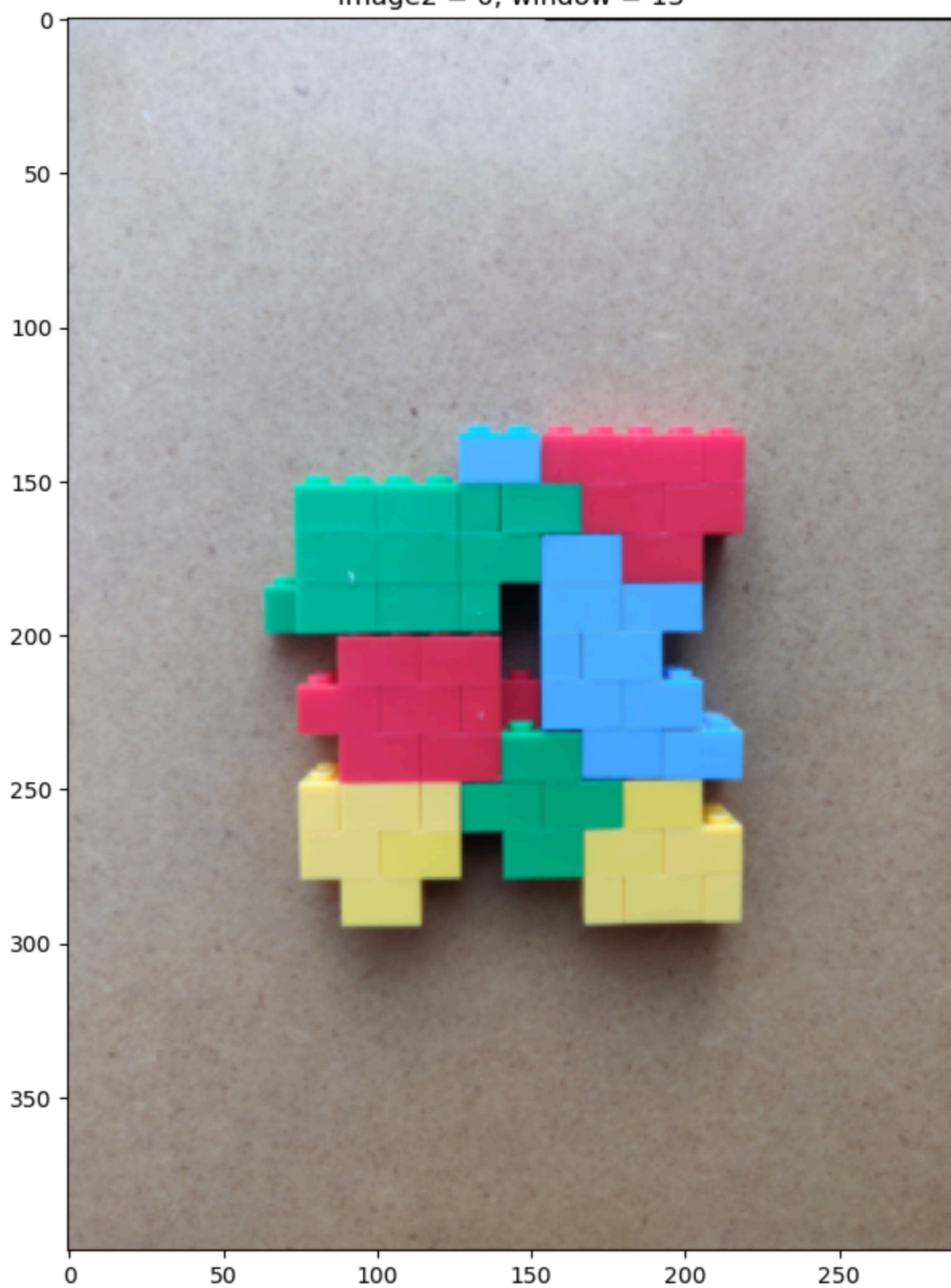


image2 = 1, window = 13

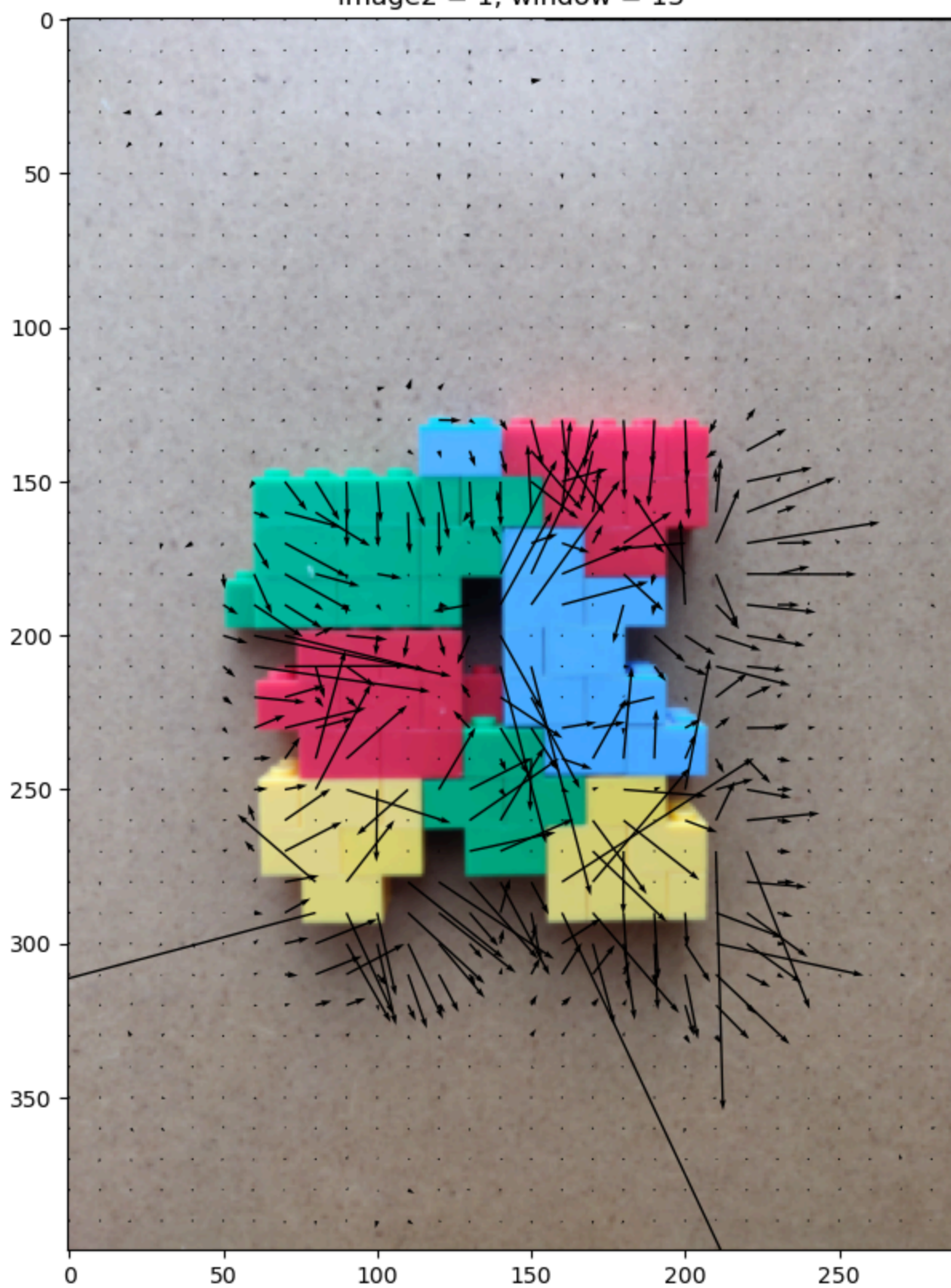
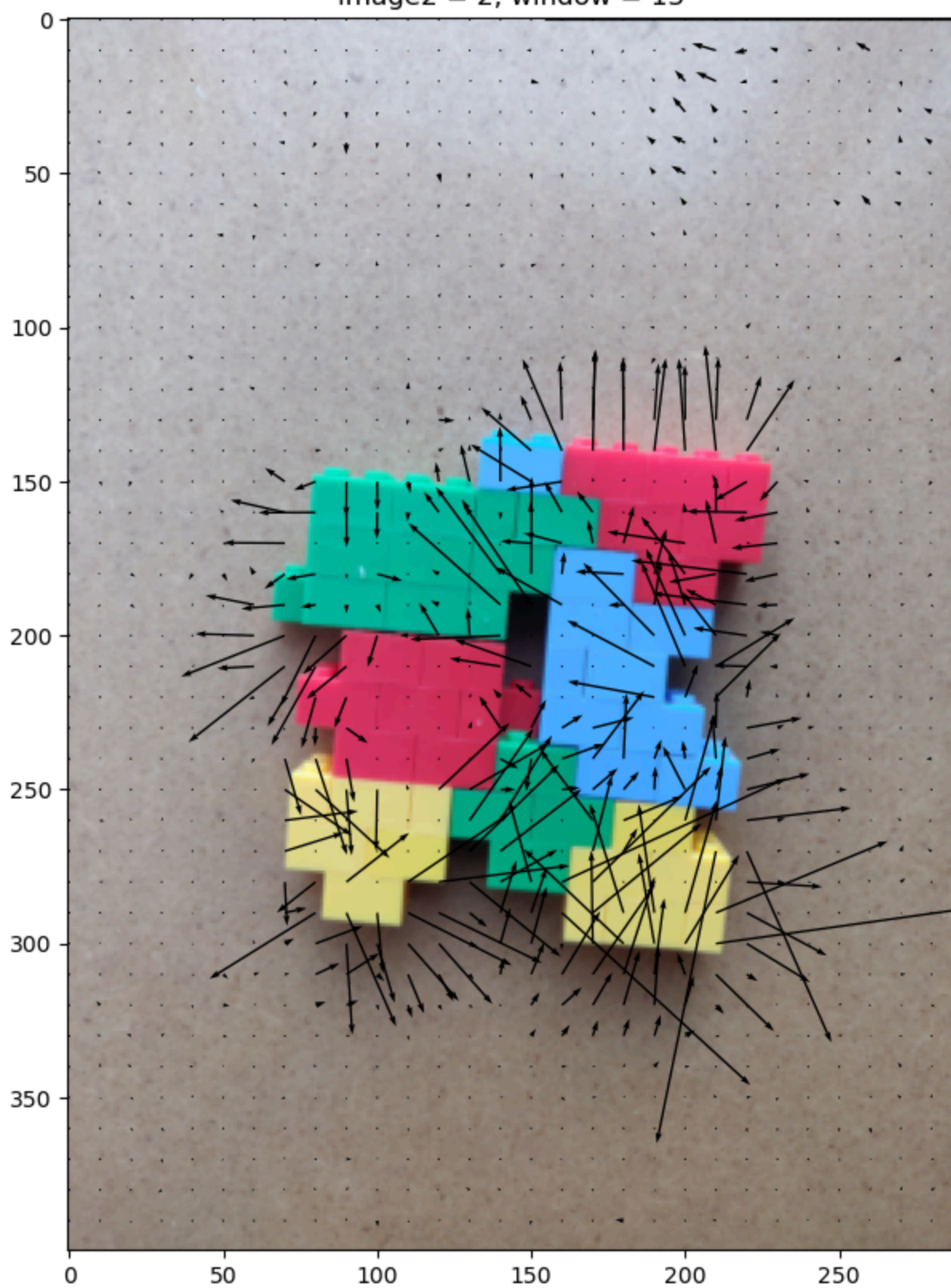
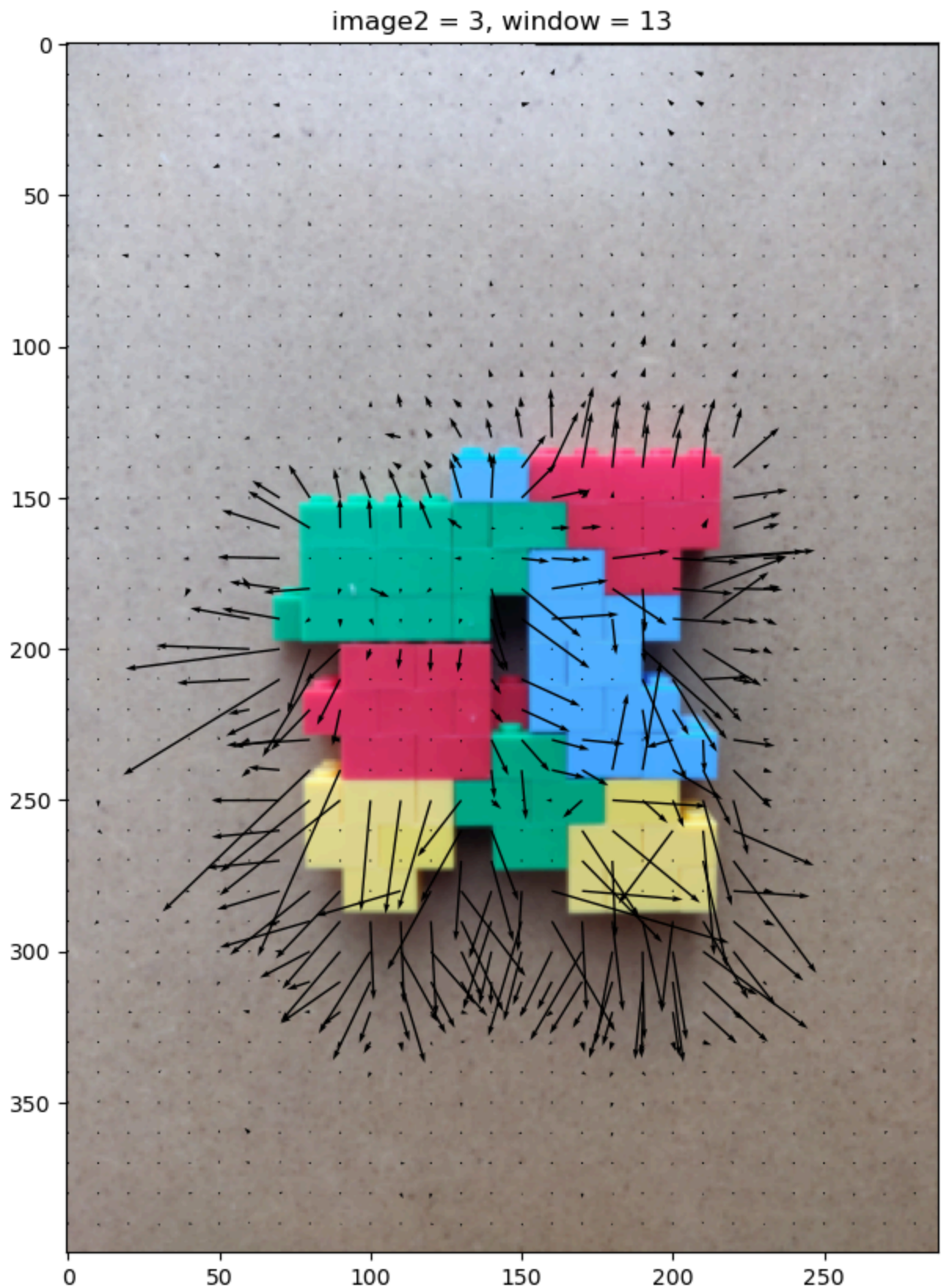


image2 = 2, window = 13





display your plots above by fully expanding the above block, or your plots here

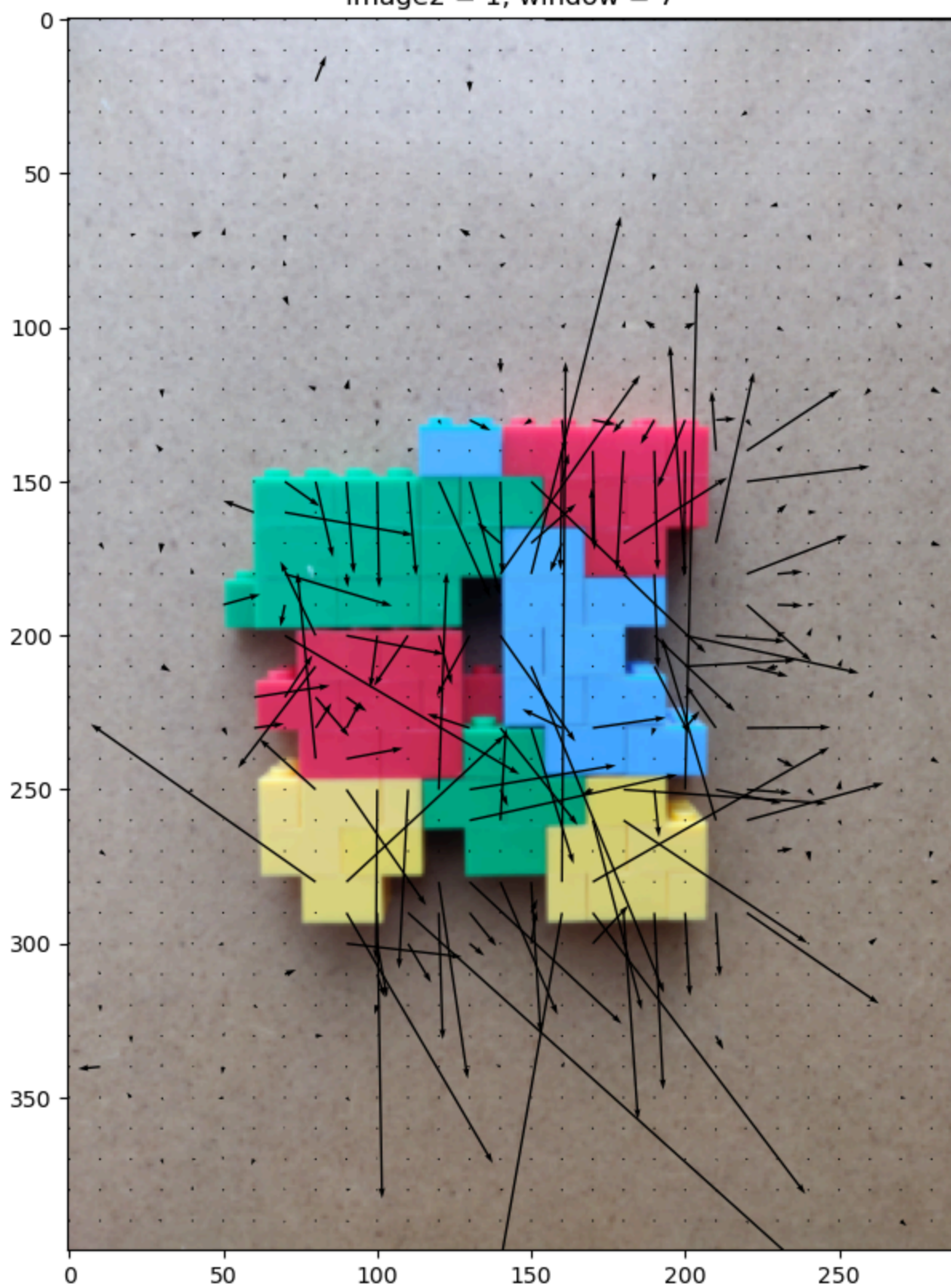
I had to change the plotting options but I do see how it can move along the vectors, but intuition wise its hard to think of it as vectors from each pixel

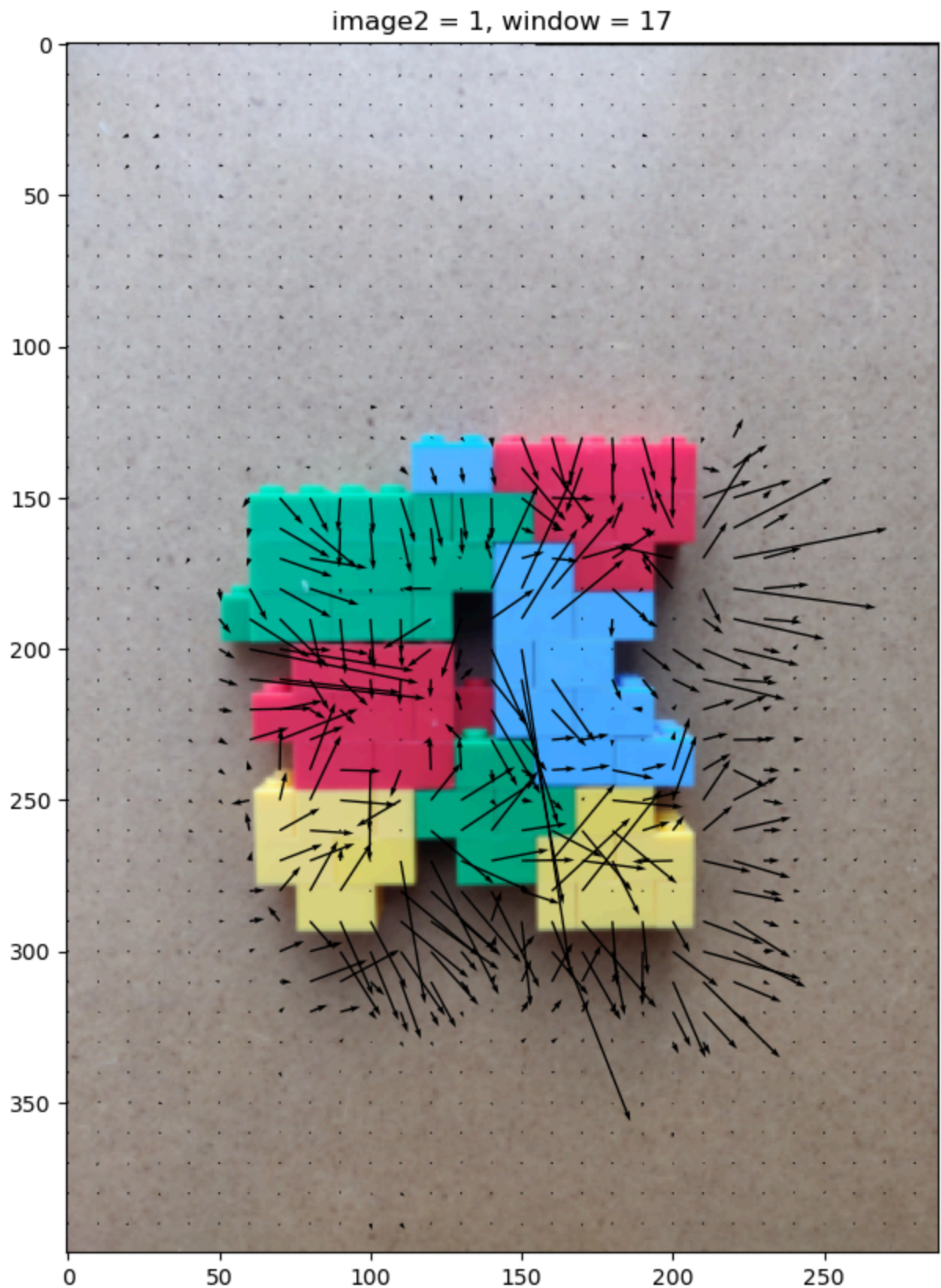
3. For the first pair of images above, estimate two new optical flows using one smaller-sized window and another larger-sized window, which lead to observable difference in the results. Comment on the effect of window size on the results and justify. **(4 points)**

```
In [10]: window = 7
U, V = LucasKanadeOpticalFlow(grayscale(images[0]),grayscale(images[1]), window)
plot_optical_flow(images[1], U, V, 'image2 = ' + str(1) + ', window = ' + str(window))

window = 17
U, V = LucasKanadeOpticalFlow(grayscale(images[0]),grayscale(images[1]), window)
plot_optical_flow(images[1], U, V, 'image2 = ' + str(1) + ', window = ' + str(window))
```

image2 = 1, window = 7





The smaller window size seems to result in larger, more "chaotic" vectors while the larger window has the opposite effect. The larger window will have more data on the flows and smooths out to be more similar to the points around as they have more overlap, being able to see more of the image.

4. Implement the incomplete lines within `def LucasKanadeOpticalFlow` in the first block to handle accumulated flow in the coarse-to-fine scheme (corresponding to `LKmode == 'multiscale'`). Then complete the `def opticalFlowMultiScale` function below and run the block to compute and visualize flow in coarse-to-fine scales. **(5 points)**

One of your output images which is at the coarsest scale should look like this (but doesn't have to be identical):

```
In [11]: from scipy.interpolate import interp2d

def upsampleFlow(U_prev, V_prev):
    x, y = U_prev.shape
    x_new = np.linspace(0, x, x*2)
    y_new = np.linspace(0, y, y*2)

    func_u = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), U_prev)
    u = func_u(y_new, x_new) * 2

    func_v = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), V_prev)
    v = func_v(y_new, x_new) * 2

    return u, v

def opticalFlowMultiScale(im1, im2, window_size, num_levels):
    im1_arr, im2_arr = [], []
    u_list, v_list = [], []
    for i in range(num_levels):
        im1_arr.insert(0, im1)
        im2_arr.insert(0, im2)
        im1 = im1[::2, ::2]
        im2 = im2[::2, ::2]

    print("Level 0: image_shape: ", im1_arr[0].shape)
    #u, v = LucasKanadeOpticalFlow( # YOUR CODE HERE: compute flow in the coarsest s
    u, v = LucasKanadeOpticalFlow(im1_arr[0], im2_arr[0], window_size, LKmode='none')
    u_list.append(u)
    v_list.append(v)

    for i in range(1, num_levels):
        print("Level {}: image_shape: {}".format(i, im1_arr[i].shape))
        ##### (Q4.4) YOUR CODE HERE #####
        # upsample previous flow
        u_upsampled, v_upsampled = upsampleFlow(u_list[i-1], v_list[i-1])
        # use the image pair from current level i, along with upsampled flow as inp
        u_i, v_i = LucasKanadeOpticalFlow(im1_arr[i], im2_arr[i], window_size,
                                          u_prev=u_upsampled, v_prev=v_upsampled,
                                          LKmode='multiscale')
        #####
        u_list.append(u_i)
        v_list.append(v_i)
```

```

    return u_list, v_list, im1_arr, im2_arr

images = []
for i in range(1,5):
    images.append(plt.imread('OpticalFlowImages/im'+str(i)+'.png')[:, :288, :])
# each image after converting to gray scale is of size -> 400x288

window = 13
U_list, V_list, im1_arr, im2_arr = opticalFlowMultiScale( grayscale(images[0]), grayscale(images[1]), window)

for l, (U, V, im1) in enumerate(zip(U_list, V_list, im1_arr)):
    print(U.shape, im1.shape)
    plot_optical_flow(im1, U, V, 'level = ' + str(l) + ' image2 = ' + str(3) + ', w

```

Level 0: image_shape: (100, 72)

Level 1: image_shape: (200, 144)

/tmp/ipykernel_270/303067314.py:8: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline` on regular grids, and `bisplrep`/`bisplev` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator` instead. For scattered data, prefer `LinearNDInterpolator` or `CloughTocher2DInterpolator`.

For more details see

[`https://scipy.github.io/devdocs/notebooks/interp\\_transition\\_guide.html`](https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html)

```
func_u = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), U_prev)
```

/tmp/ipykernel_270/303067314.py:9: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline` on regular grids, and `bisplrep`/`bisplev` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator` instead. For scattered data, prefer `LinearNDInterpolator` or `CloughTocher2DInterpolator`.

For more details see

[`https://scipy.github.io/devdocs/notebooks/interp\\_transition\\_guide.html`](https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html)

```
u = func_u(y_new, x_new) * 2
```

/tmp/ipykernel_270/303067314.py:11: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline` on regular grids, and `bisplrep`/`bisplev` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator` instead. For scattered data, prefer `LinearNDInterpolator` or `CloughTocher2DInterpolator`.

For more details see

[`https://scipy.github.io/devdocs/notebooks/interp\\_transition\\_guide.html`](https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html)

```
func_v = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), V_prev)
```

/tmp/ipykernel_270/303067314.py:12: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline` on regular grids, and `bisplrep`/`bisplev` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator` instead. For scattered data, prefer `LinearNDInterpolator` or `CloughTocher2DInterpolator`.

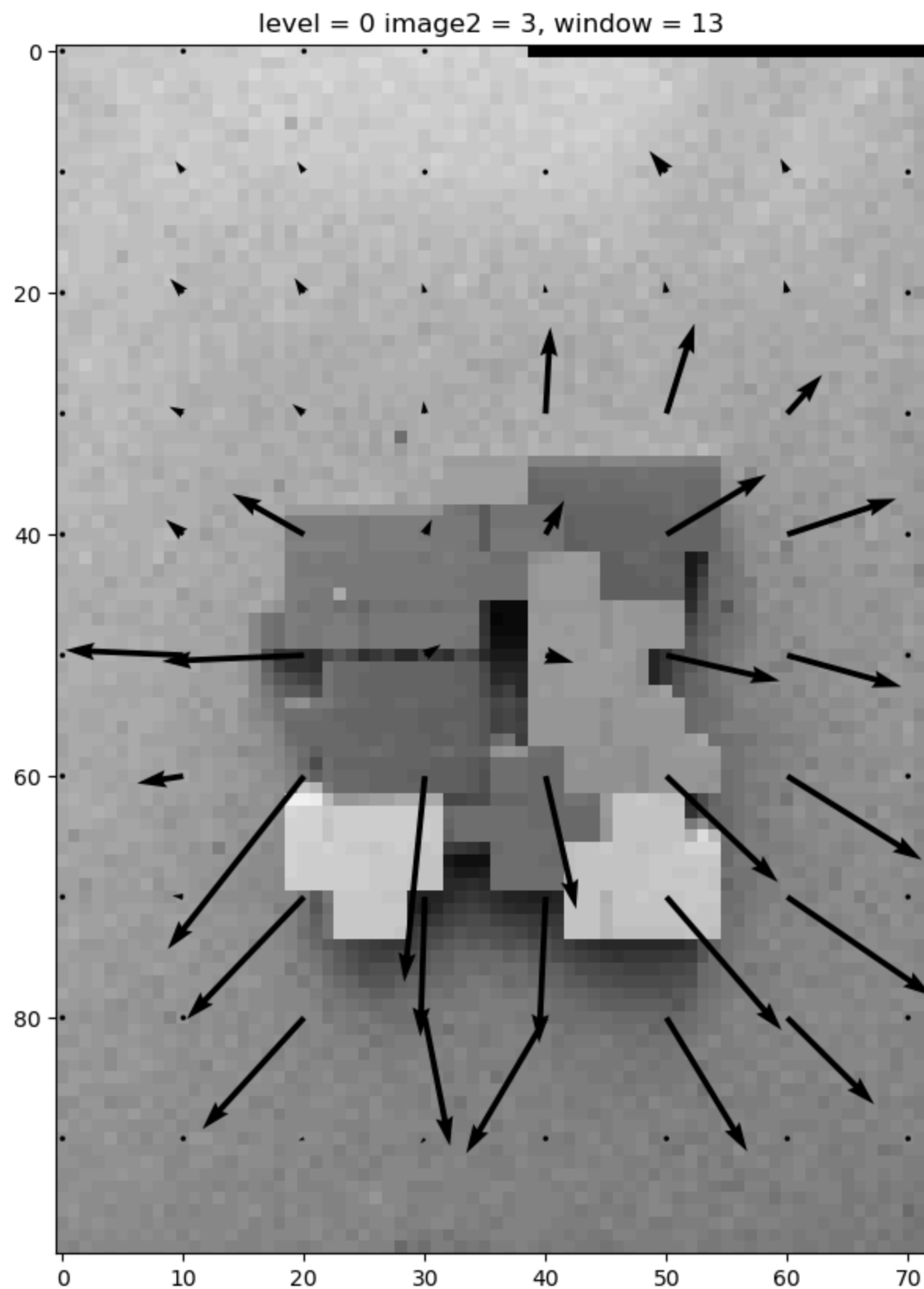
For more details see

[https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html`](https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html)

```
v = func_v(y_new, x_new) * 2
```

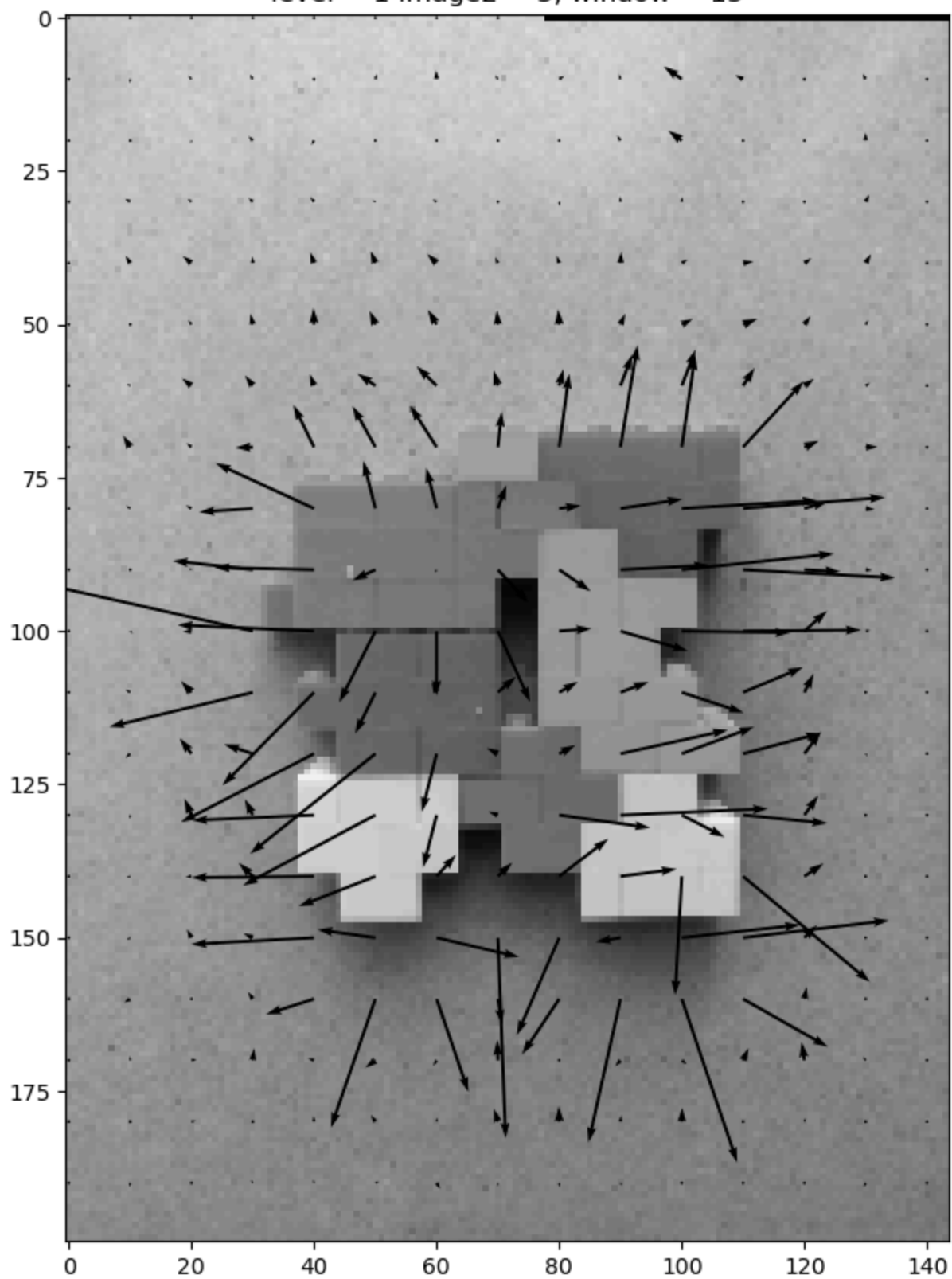
Level 2: image_shape: (400, 288)

(100, 72) (100, 72)

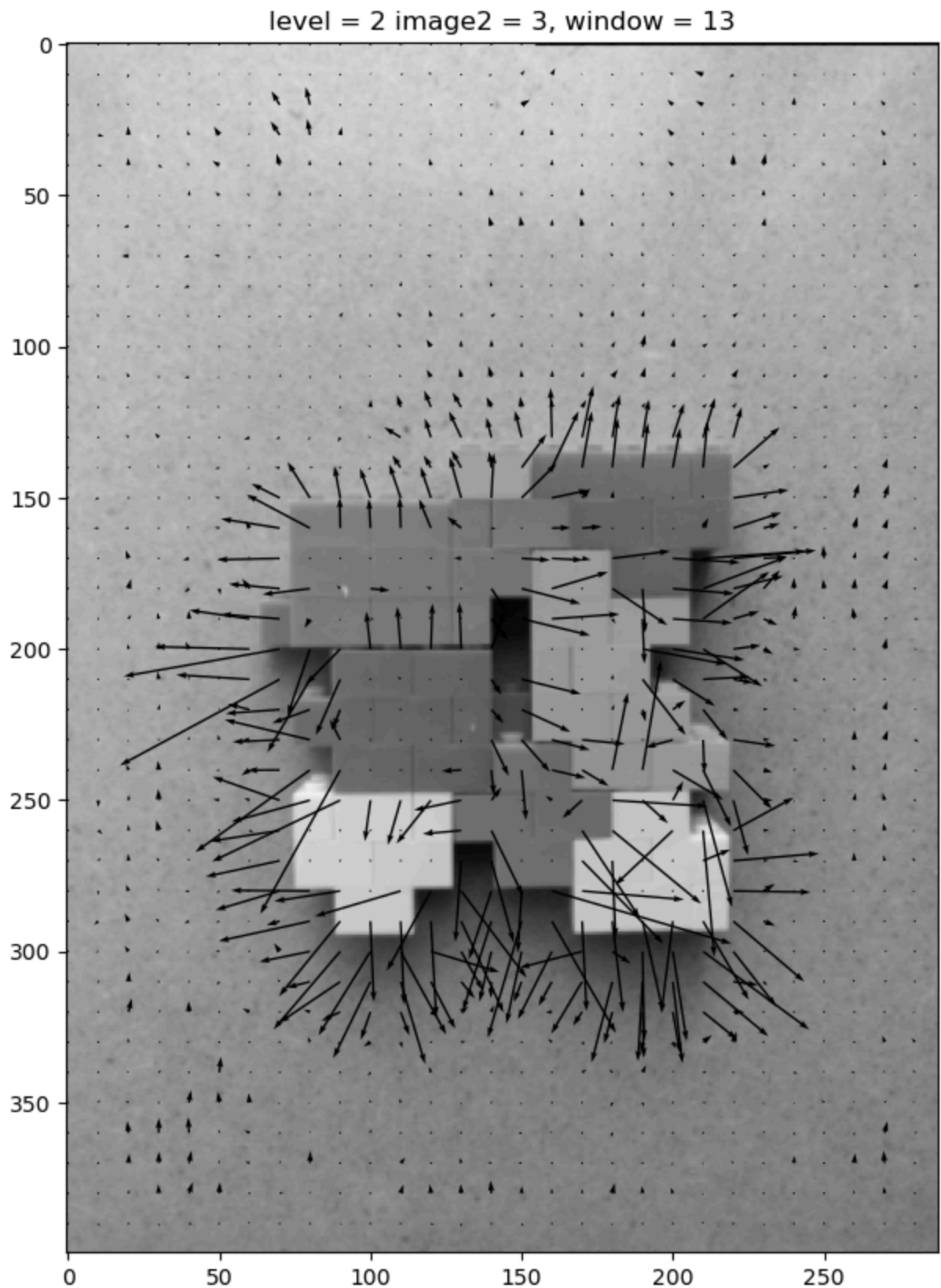


(200, 144) (200, 144)

level = 1 image2 = 3, window = 13



(400, 288) (400, 288)



5. We will now consider two pairs of KITTI images located at `/datasets/cs152b-sp22-a00-public/dataset_SfM/sequences/00/image_0`, from a video sequence recorded while driving down a street.

First consider the image pair 000000.png and 000001.png shown below. Compute the coarse-to-fine optical flow and plot the output at the finest scale.

Next, consider the image pair 000106.png and 000107.png shown below. Compute the coarse-to-fine optical flow and plot the output at the finest scale.

Now describe the pattern of flow vectors in the above two examples and explain how each of them relates to the type of camera motion between each pair of images. **(6 points)**

```
In [28]: images = []
images.append(plt.imread('dataset_SfM/sequences/00/image_0/000000.png')[:, :400])
images.append(plt.imread('dataset_SfM/sequences/00/image_0/000001.png')[:, :400])
# each image after converting to gray scale is of size -> 400x288

window = 25
U_list, V_list, im1_arr, im2_arr = opticalFlowMultiScale(images[0], images[1], window)

for l, (U, V, im1) in enumerate(zip(U_list, V_list, im1_arr)):
    print(U.shape, im1.shape)
    plot_optical_flow(im1, U, V, 'level = ' + str(l) + ' image2 = ' + str(3) + ', w
```

Level 0: image_shape: (94, 100)

Level 1: image_shape: (188, 200)

/tmp/ipykernel_270/303067314.py:8: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
func_u = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), U_prev)
```

/tmp/ipykernel_270/303067314.py:9: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
u = func_u(y_new, x_new) * 2
```

/tmp/ipykernel_270/303067314.py:11: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
func_v = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), V_prev)
```

/tmp/ipykernel_270/303067314.py:12: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

[https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html`](https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html)

```
v = func_v(y_new, x_new) * 2
```

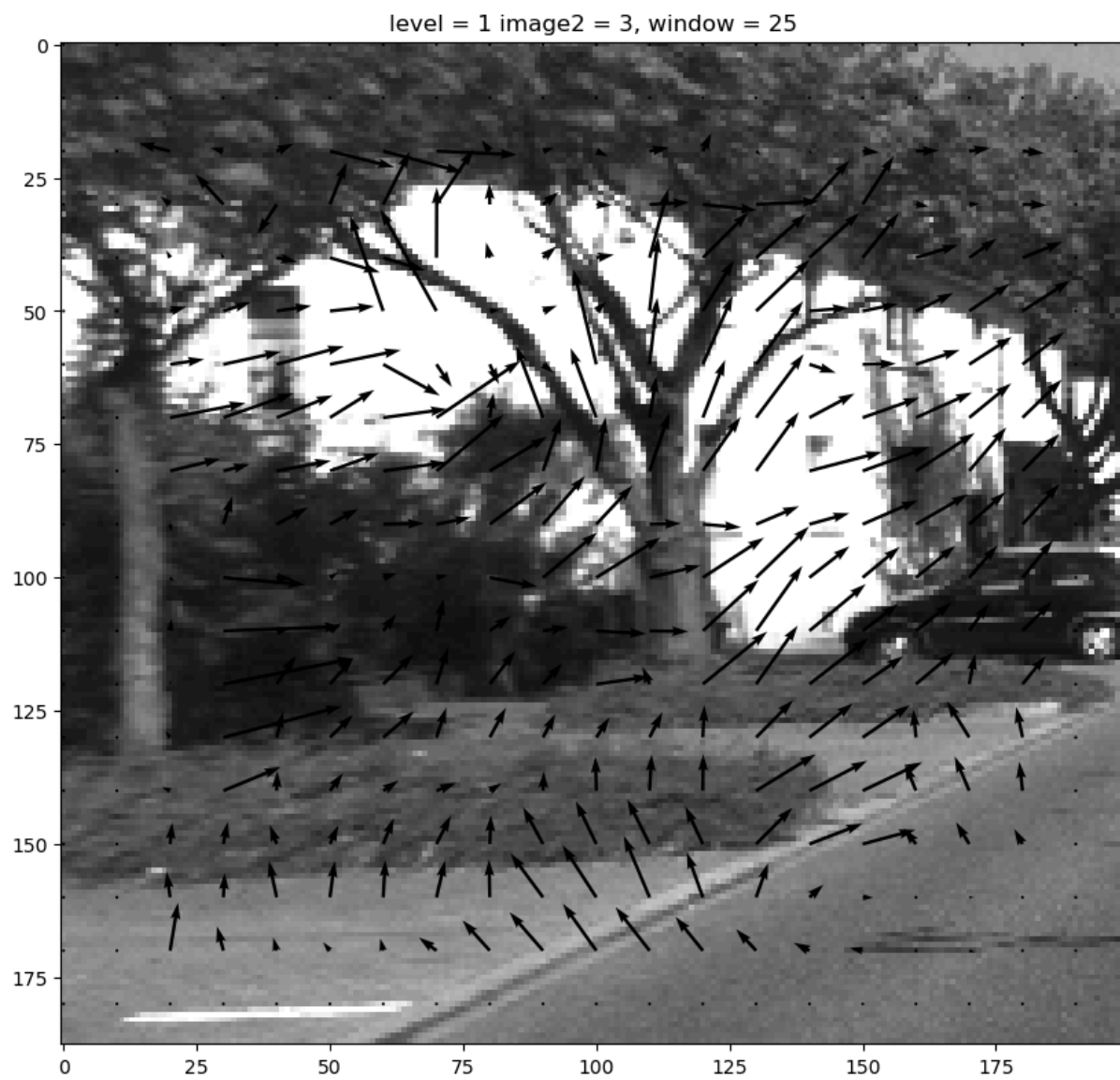
Level 2: image_shape: (376, 400)

(94, 100) (94, 100)

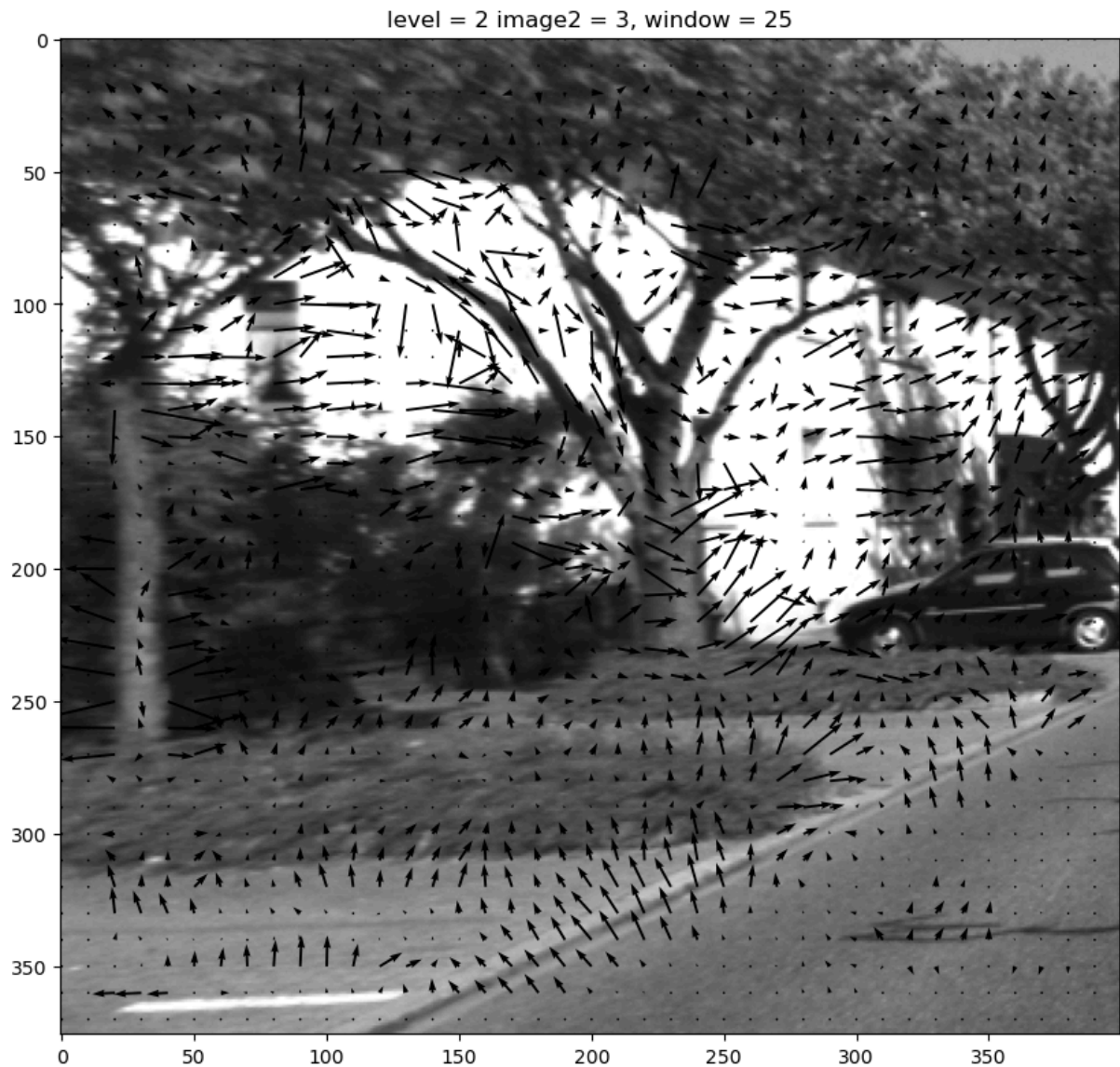
level = 0 image2 = 3, window = 25



(188, 200) (188, 200)



(376, 400) (376, 400)



```
In [23]: images = []
images.append(plt.imread('dataset_SfM/sequences/00/image_0/000106.png')[:, :400])
images.append(plt.imread('dataset_SfM/sequences/00/image_0/000107.png')[:, :400])
# each image after converting to gray scale is of size -> 400x400

window = 25
U_list, V_list, im1_arr, im2_arr = opticalFlowMultiScale(images[0], images[1], window)

for l, (U, V, im1) in enumerate(zip(U_list, V_list, im1_arr)):
    print(U.shape, im1.shape)
    plot_optical_flow(im1, U, V, 'level = ' + str(l) + ' image2 = ' + str(3) + ', w
```

Level 0: image_shape: (94, 100)

Level 1: image_shape: (188, 200)

/tmp/ipykernel_270/303067314.py:8: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
func_u = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), U_prev)
```

/tmp/ipykernel_270/303067314.py:9: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
u = func_u(y_new, x_new) * 2
```

/tmp/ipykernel_270/303067314.py:11: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
func_v = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), V_prev)
```

/tmp/ipykernel_270/303067314.py:12: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

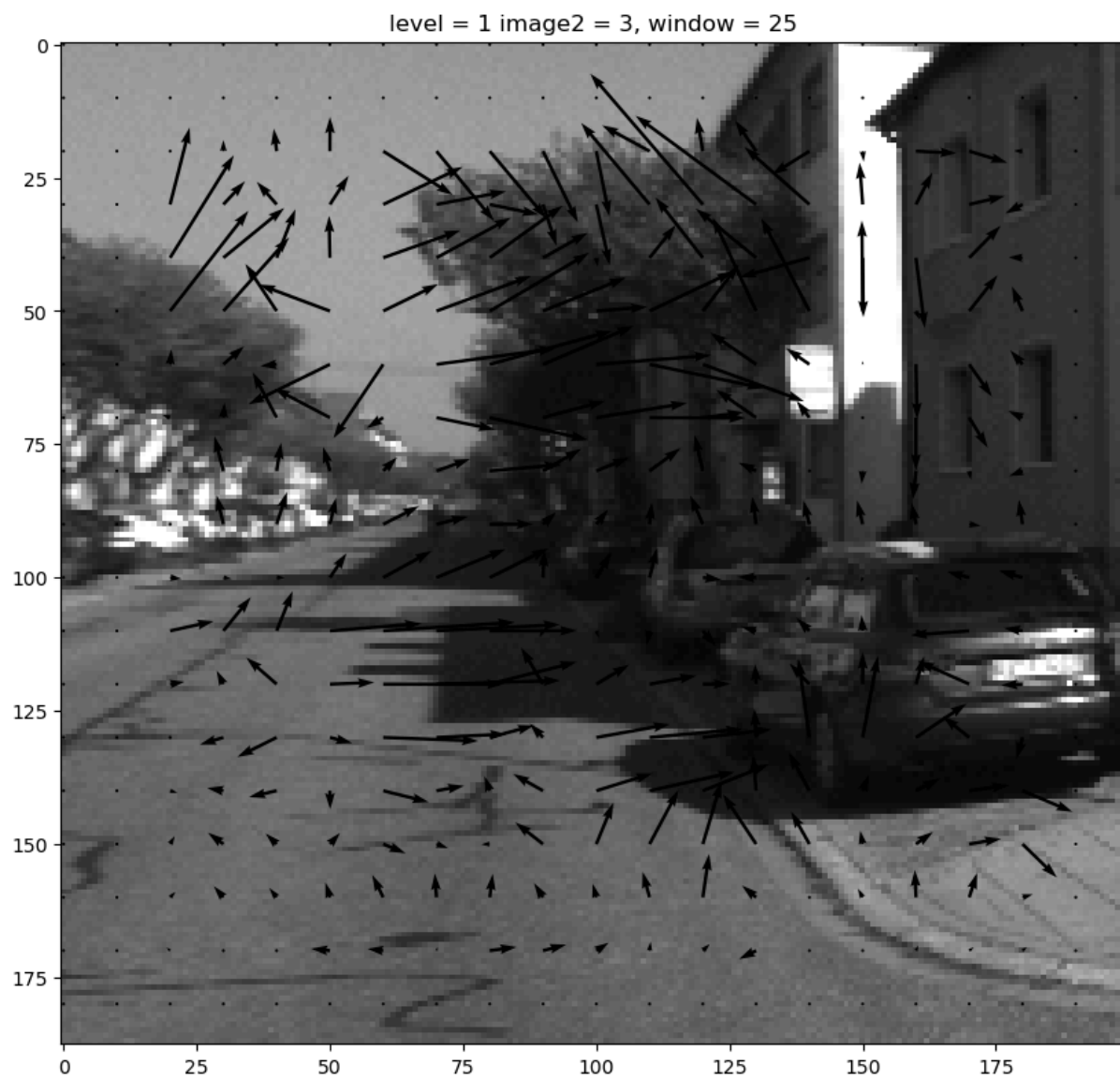
For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

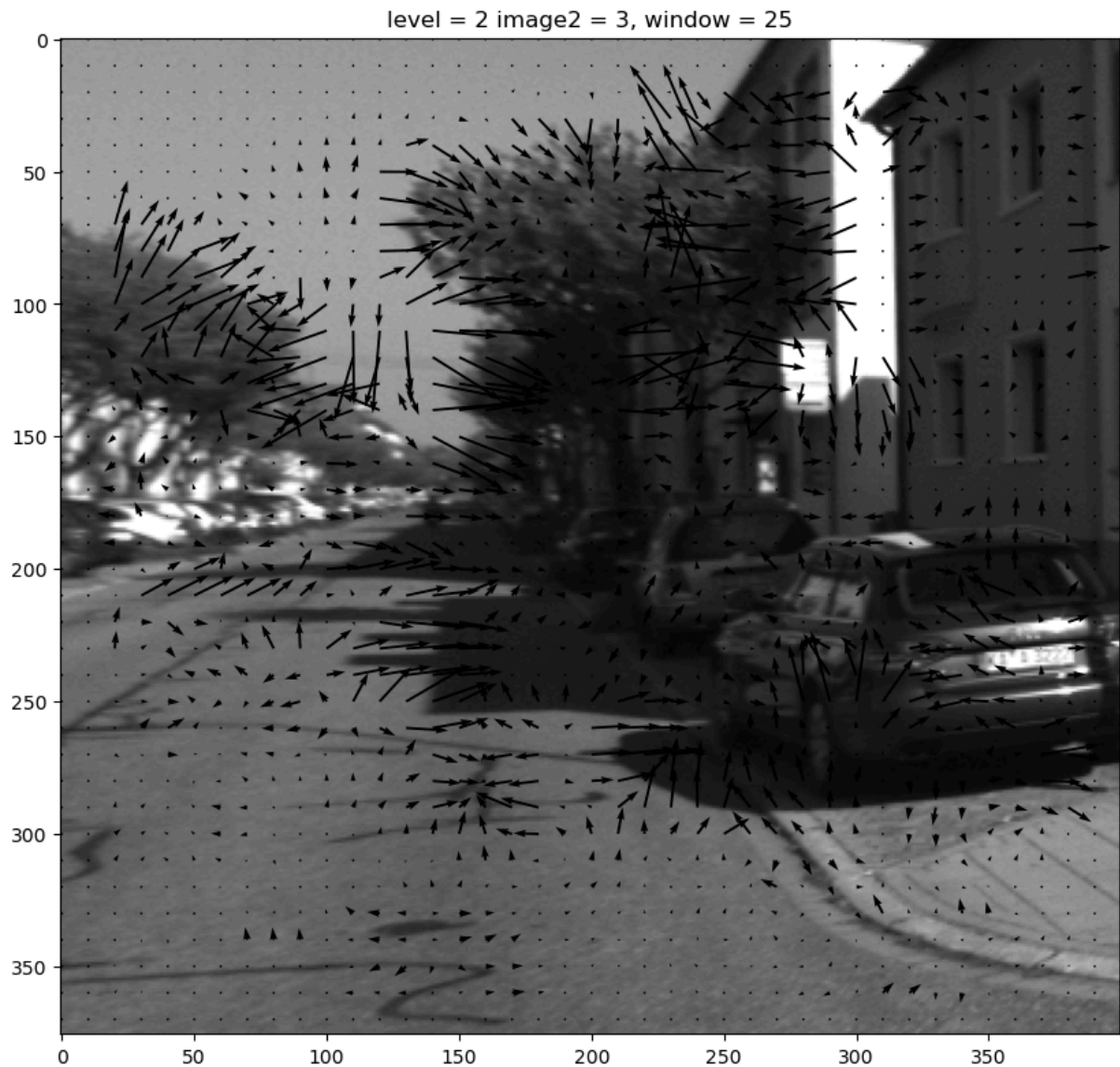
For more details see
[https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html`](https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html)

```
v = func_v(y_new, x_new) * 2  
Level 2: image_shape: (376, 400)  
(94, 100) (94, 100)
```





(376, 400) (376, 400)



The flow patterns follow the idea with the window size, as more data and more pixels are considered, the motion vectors get smaller and more aligned to explain a more cohesive image

6. Now, use a camera to take two images of your own with very small motion (for example, consecutive frames of a video sequence). Compute the coarse-to-fine optical flow and comment on whether the flow vectors are consistent with what you expected given the motion. **(4 points)**

You may need to resize the two images to the same size, where both height and width are divisible by 4 (as required by the coarse-to-fine code). Feel free to adjust the window size and number of scales to achieved better results.

```
In [ ]: images = []
images.append(plt.imread('images/wall1.jpg')[:400,:400])
images.append(plt.imread('images/wall2.jpg')[:400,:400])
```

```
window = 25
U_list, V_list, im1_arr, im2_arr = opticalFlowMultiScale( grayscale(images[0]), grayscale(images[1]), window)

for l, (U, V, im1) in enumerate(zip(U_list, V_list, im1_arr)):
    print(U.shape, im1.shape)
    plot_optical_flow(im1, U, V, 'level = ' + str(l) + ' image2 = ' + str(3) + ', window = ' + str(window))
```

Level 0: image_shape: (1008, 756)

Level 1: image_shape: (2016, 1512)

/tmp/ipykernel_270/303067314.py:8: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
func_u = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), U_prev)
```

/tmp/ipykernel_270/303067314.py:9: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
u = func_u(y_new, x_new) * 2
```

/tmp/ipykernel_270/303067314.py:11: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
func_v = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), V_prev)
```

/tmp/ipykernel_270/303067314.py:12: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
v = func_v(y_new, x_new) * 2
```

Level 2: image_shape: (4032, 3024)

```
In [35]: images = []
images.append(plt.imread('images/wall1.jpg')[:400,:400])
images.append(plt.imread('images/wall2.jpg')[:400,:400])

window = 25
U_list, V_list, im1_arr, im2_arr = opticalFlowMultiScale( grayscale(images[0]), grayscale(images[1]), window)

for l, (U, V, im1) in enumerate(zip(U_list, V_list, im1_arr)):
    print(U.shape, im1.shape)
    plot_optical_flow(im1, U, V, 'level = ' + str(l) + ' image2 = ' + str(3) + ', w
```

Level 0: image_shape: (100, 100)

Level 1: image_shape: (200, 200)

/tmp/ipykernel_270/303067314.py:8: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
func_u = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), U_prev)
```

/tmp/ipykernel_270/303067314.py:9: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
u = func_u(y_new, x_new) * 2
```

/tmp/ipykernel_270/303067314.py:11: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

For more details see

``https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html``

```
func_v = interp2d(np.array(np.arange(y)), np.array(np.arange(x)), V_prev)
```

/tmp/ipykernel_270/303067314.py:12: DeprecationWarning: `interp2d` is deprecated in SciPy 1.10 and will be removed in SciPy 1.14.0.

For legacy code, nearly bug-for-bug compatible replacements are `RectBivariateSpline`` on regular grids, and `bisplrep`/bisplev`` for scattered 2D data.

In new code, for regular grids use `RegularGridInterpolator`` instead. For scattered data, prefer `LinearNDInterpolator`` or `CloughTocher2DInterpolator``.

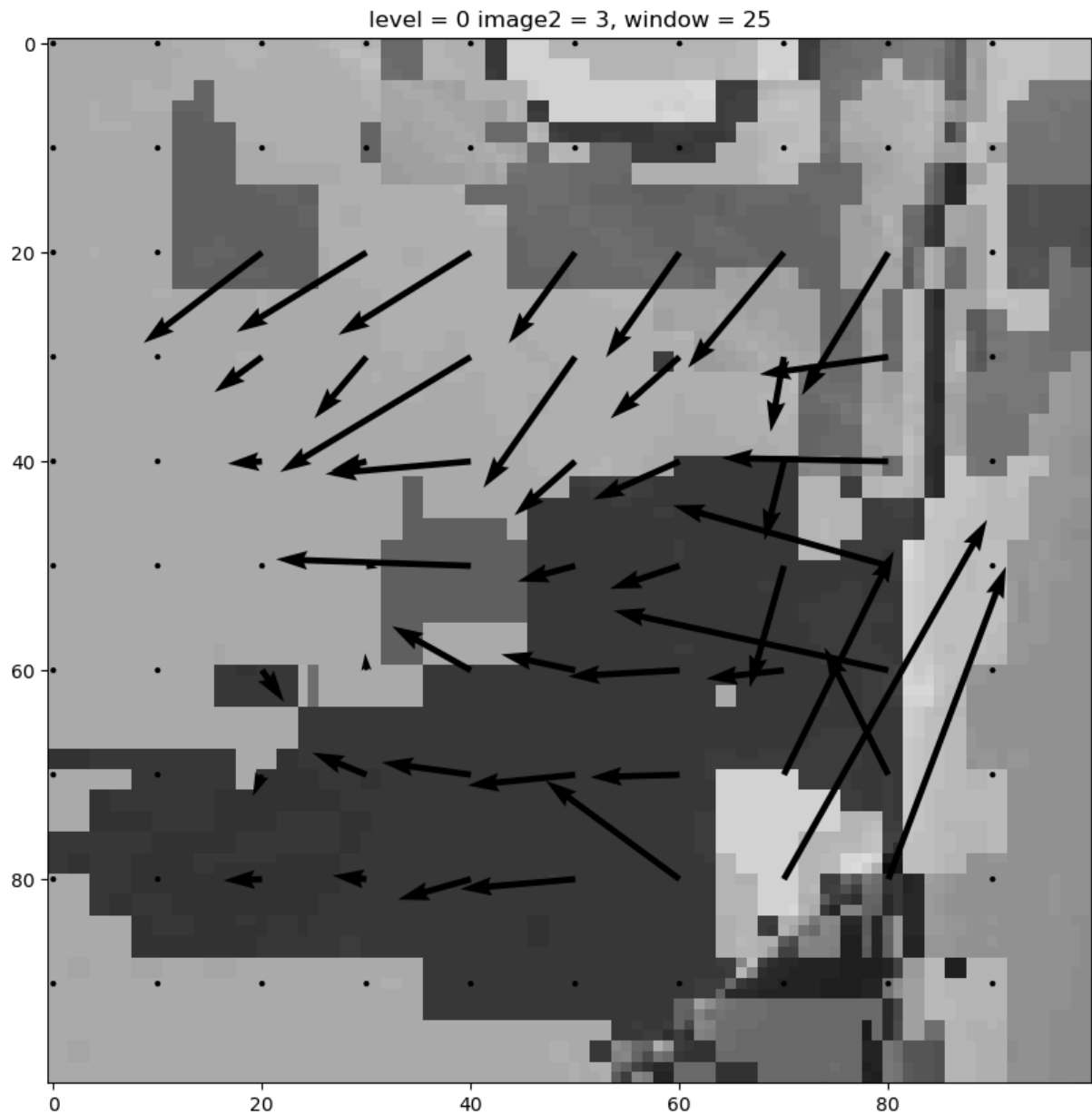
For more details see

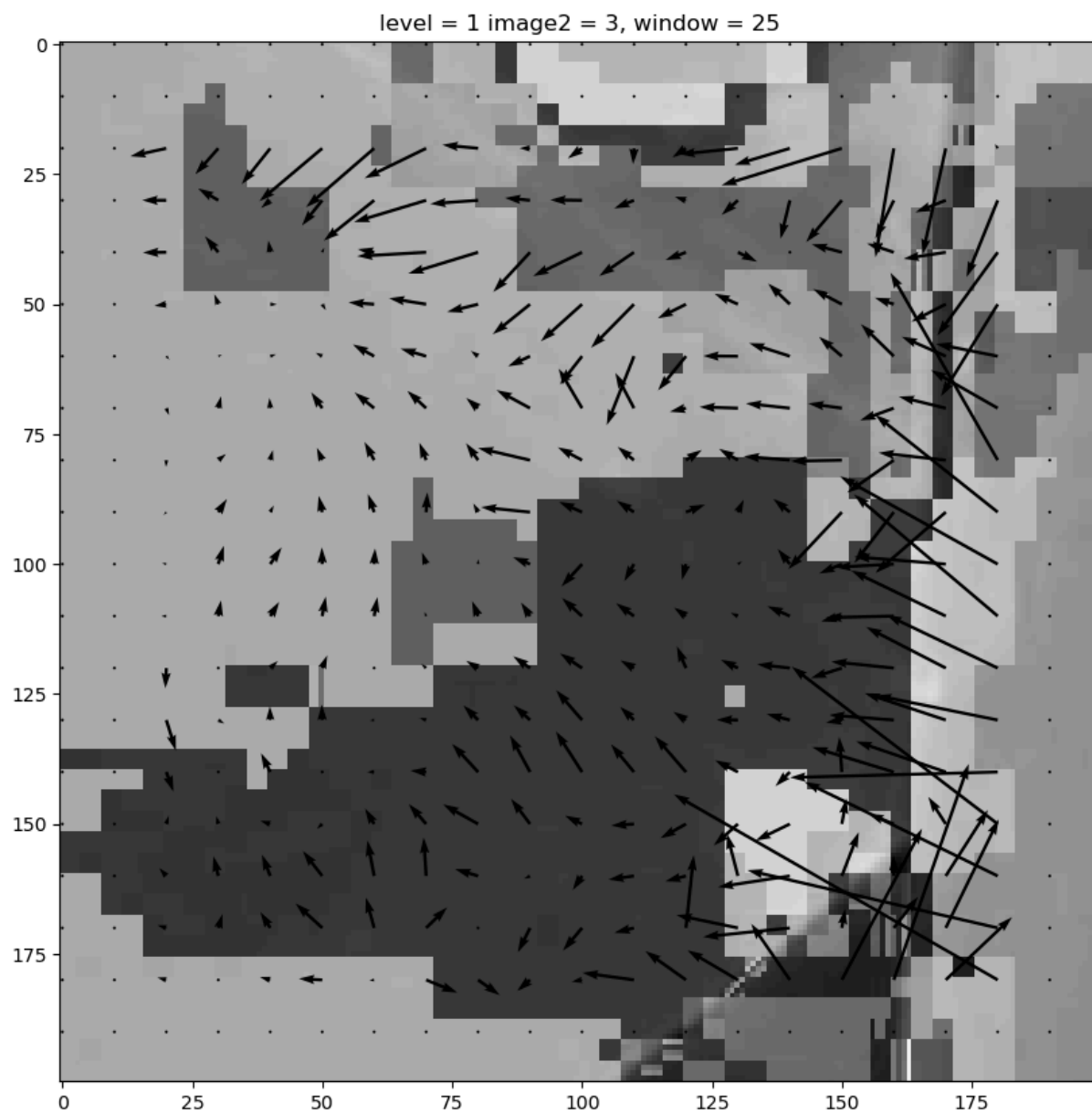
[https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html`](https://scipy.github.io/devdocs/notebooks/interp_transition_guide.html)

```
v = func_v(y_new, x_new) * 2
```

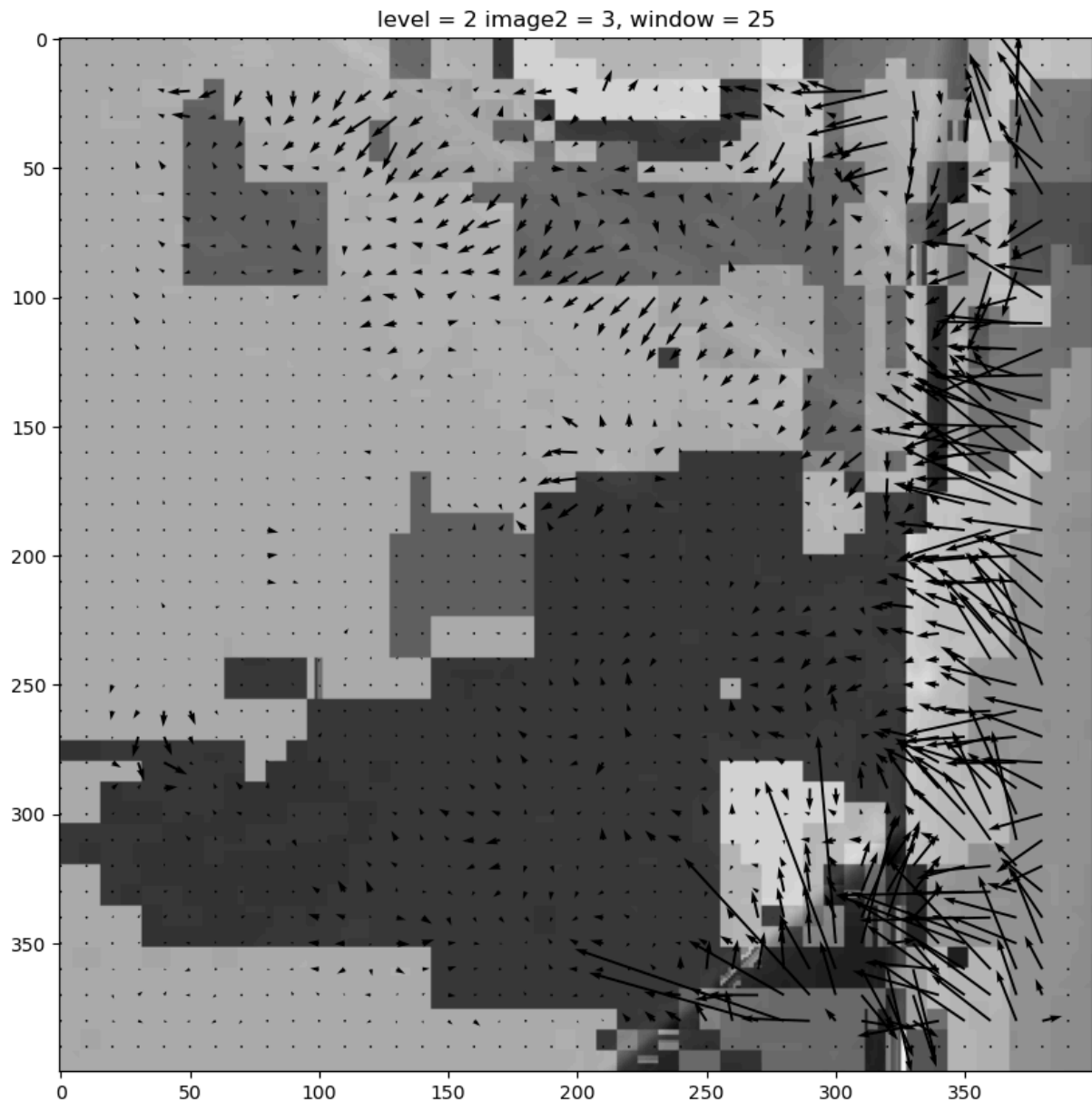
Level 2: image_shape: (400, 400)

(100, 100) (100, 100)





(400, 400) (400, 400)



The optical flow estimations here mirror the process for the optical flow of the previous images, these images were much larger because they had higher resolution, but the process remains the same. The second one (which actually finished running, is a bit pixelated but helps to show how the flow is calculated

References

1. Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superpoint: Self-supervised interest point detection and description. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops, pages 224–236, 2018.
2. Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In Conference on Computer Vision and Pattern Recognition (CVPR), 2012.

3. Andreas Geiger, Julius Ziegler, and Christoph Stiller. Stereoscan: Dense 3d reconstruction in real-time. In Intelligent Vehicles Symposium (IV), 2011.
4. David G Lowe. Distinctive image features from scale-invariant keypoints. IJCV, 60(2):91–110, 2004.
5. Anurag Ranjan and Michael J Black. Optical flow estimation using a spatial pyramid network. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 4161–4170, 2017.
6. A. Vedaldi and B. Fulkerson. VLFeat: An open and portable library of computer vision algorithms. <http://www.vlfeat.org/>, 2008.
7. Lucas, Bruce D., and Takeo Kanade. "An iterative image registration technique with an application to stereo vision." (1981): 674.